FL 実験コード サポートガイド

連合学習における悪性更新の検知実験 — 基礎知識からコードの使い方まで

3CDA2216 笠原秀星

2026 年 10 月 17 日

目次

1 このガイドについて 5

第 I 部 基礎知識 5

2 機械学習と連合学習 6
2.1 機械学習の最小限 . . . 6
2.2 連合学習とは . . . 6
2.3 FedAvg（集約の方法） . . . 7
2.4 IID / Non-IID と Dirichlet 分割 . . . 7

3 攻撃と検知 8
3.1 悪性クライアントと，この実験の前提 . . . 8
3.2 更新ベクトル . . . 9
3.3 符号反転攻撃（sign flip） . . . 9
3.4 ALIE 攻撃（A Little Is Enough） . . . 9
3.5 広がり（spread）と押し出し（push） . . . 10
3.6 検知という問題 . . . 10
3.7 検知器 D1：大きさを見る（ノルム偏差） . . . 11
3.8 検知器 D2：向きを見る（コサイン距離） . . . 11
3.9 検知器 D3：まとめて見る（k-means） . . . 12
3.10 検知器 D4：一番ばらついている方向を見る（スペクトル） . . . 12
3.11 4 つの検知器のまとめ . . . 13

4 評価とスイープ 13
4.1 ROC-AUC . . . 13
4.2 そのほかの指標 . . . 14
4.3 スイープとは . . . 14

第 II 部 コードの全体像 15

5 全体像 15
5.1 ファイルの配置と，実験の流れ . . . 15
5.2 `fl_detection/` の中のつながり . . . 15
5.3 1 ラウンドのあいだに起きること . . . 16

6 各ファイルの説明 17
6.1 読む順番の目安 . . . 17
6.2 fl_detection/server_app.py（サーバの入口） . . . 17
6.3 fl_detection/client_app.py（クライアントの入口） . . . 18
6.4 fl_detection/robust_strategy.py（毎ラウンドの進行役） . . . 18
6.5 fl_detection/attack.py（攻撃の中身） . . . 18
6.6 fl_detection/detection.py（検知器と成績） . . . 19
6.7 fl_detection/task.py（モデル・データ・学習） . . . 19
6.8 fl_detection/aggregation.py（形の変換と集約） . . . 19
6.9 fl_detection/malicious.py（誰が悪性か） . . . 20
6.10 fl_detection/log_output.py（ログを書く） . . . 20
6.11 run_experiments.py（スイープの起動） . . . 20
6.12 analyze_detection.py（結果の分析） . . . 20

第 III 部 使い方 20

7 環境の構築 21
7.1 必要なもの . . . 21
7.2 手順 . . . 21
7.3 GPU を使うための設定 . . . 22
7.4 クライアントの台数の設定 . . . 23
7.5 初回実行時の注意（MNIST のダウンロード） . . . 23

8 設定（pyproject.toml） 23
8.1 設定はここだけに書く . . . 23
8.2 共通の設定 . . . 24
8.3 スイープの軸 . . . 24

9 実行手順 25
9.1 手順 1：テスト . . . 25
9.2 手順 2：条件の確認 . . . 25
9.3 手順 3：動作確認（1 条件だけ短く回す） . . . 25
9.4 手順 4：スイープ . . . 25
9.5 手順 5：分析 . . . 26

10 出力ファイル 26
10.1 detection_updates.csv（1 クライアント 1 行） . . . 26
10.2 detection_metrics.csv（1 検知器 1 行） . . . 27
10.3 分析の出力（analysis/） . . . 28

第 IV 部 卒論の主張と根拠 28

11 この部の使い方 28

12 図と表の読み方 29
12.1 精度の推移（accuracy_by_round.png）と精度の表（accuracy_table.csv） . . . 30
12.2 最終 AUC の表（卒論の表 tab:final-auc，detection_final_auc_table.csv） . . . 30

12.3 z と AUC の図（detection_auc_by_z.png） . . . 30
12.4 AUC のラウンド推移（detection_auc_by_round.png） . . . 31
12.5 push_ratio と AUC の散布図（detection_pushratio_vs_auc.png）と相関の表 . . . 31
12.6 spread・push_ratio の表と，分布構造の表（detection_structure_table.csv） . . . 32

13 結果の全体像 33

14 主張：精度と収束 33

15 主張：D2 は ALIE を逆向きに完全に分ける 33

16 主張：D3 はいつも高いが，理由に注意 34

17 主張：重心に近すぎる悪性も目立つ（高次元の殻） 34

18 主張：D4 の境界は Non-IID で動く（分布の形） 35

19 主張：中心仮説は支持されなかった（符号反転と push_ratio） 36

20 主張：符号反転を D1 が見つけられる理由 36

21 主張：良性の各座標は，裾が正規分布より重い 37

22 主張：ALIE は FedAvg ではほぼ無害 37

23 事前の予測と結果 37

24 数値を自分で計算し直す 37

25 審査で聞かれそうなことと答え方 38

第 V 部　リファレンス 38

26 攻撃と検知器の早見表 39
26.1 攻撃 . . . 39
26.2 検知器 . . . 39

27 エラーで止まる条件 39

28 再現性 40

29 トラブルシューティング 41

30 設定を決めた経緯と，結果を読むときの注意 41
30.1 決めたことの一覧 . . . 42
30.2 α の定義を Hsu らの式に合わせた . . . 42
30.3 ローカルエポックを 2 にした . . . 43
30.4 結果を読むときの注意 . . . 43
30.5 原論文との違いのまとめ . . . 44
30.6 本番の前に確かめたこと . . . 44
30.7 あとで直すことにしたもの . . . 44

31 本書が扱わないもの 44

32 用語集 45

1　このガイドについて

本書は，「Federated Learning における悪性更新の検知可能性」を調べる実験コードの，サポートガイドである。卒業論文とは別に，**これ 1 冊で，背景の考え方からコードの動かし方までが分かる**ことを目指している。連合学習や Python にまだ慣れていない人が読むことを想定し，専門用語は出てきたところで説明する。

実験の目的・仮説・評価指標の正式な定義は，研究計画書（`sotsuron_plan2.tex`）にある。本書は，それを理解して手を動かすための手引きである。

構成。本書は，次の 5 つの部からなる。

部	内容
第 I 部　基礎知識	機械学習，連合学習，FedAvg，Non-IID，攻撃，検知器，評価指標，スイープとは何か。コードを読む前の予備知識。
第 II 部　コードの全体像	ファイルの配置，ファイルどうしの関係，1 ラウンドの流れ，各ファイルの役割。
第 III 部　使い方	環境の構築，設定，実行の手順，出力されるファイルの読み方。
第 IV 部　卒論の主張と根拠	卒論（`Memo/soturon2.tex`）の主張ごとに，根拠のログ，計算方法，確かさの度合い，審査で聞かれそうなことをまとめる。
第 V 部　リファレンス	攻撃と検知器の早見表，エラーで止まる条件，再現性，トラブルシューティング，設定を決めた経緯と結果を読むときの注意（§30），用語集。

読み方。

- 初めて読む人は，第 I 部から順に読む。
- 連合学習を知っている人は，§3（攻撃と検知）と第 II 部の 3 つの図から読む。
- 動かしたいだけの人は，第 III 部だけで足りる。
- 卒論の中身を吟味したい人は，第 IV 部（§11）から読む。主張ごとに根拠と確かさをまとめてある。
- 卒論の図や表の見方が分からないときは，§12（図と表の読み方）を見る。
- 分からない言葉が出てきたら，巻末の用語集（§32）を見る。

コードの各行の意味は，各ファイルの中にコメントとして書いてある。本書は，その入口にあたる。

この実験が調べていること。

- データの偏り（Non-IID 度）と攻撃の隠れ方を変えながら，サーバが，クライアントから届く更新だけを見て，悪性クライアントを見分けられるかを測る。
- 集約の方法は FedAvg に固定する。
- 検知した結果は記録するだけで，集約には使わない（測っているのは「検知できるか」であり，「検知して取り除いたらどうなるか」ではない）。

第 I 部
基礎知識

この部では，コードを読む前に知っておきたい考え方を，前提知識なしで説明する。すでに知っている節は飛ばしてよい。用語は巻末の用語集（§32）にもまとめてある。

2　機械学習と連合学習

2.1　機械学習の最小限

モデルと重み。 **モデル**は，入力を受け取って答えを返す計算の手順である。本実験のモデルは，手書きの数字の画像（縦 28 × 横 28 の白黒）を受け取り，それが 0〜9 のどれかを答える。モデルの中には，調整できる数値がたくさん入っている。これを**重み**（パラメータ）と呼び，まとめて w と書く。本実験のモデルでは，重みは 266,060 個ある。重みの値を変えると，モデルの答えが変わる。

学習。 **学習**とは，正解の分かっているデータを使って，モデルがなるべく正しく答えるように重みを調整することである。どれだけ外れているかを 1 つの数値で表したものを**損失**（loss）と呼び，$\ell(w)$ と書く。損失が小さいほど，よく合っている。

勾配降下法。 損失を小さくするには，「各重みを少し増やしたら損失が増えるか減るか」を調べ，損失が減る向きに重みを動かせばよい。この「増減の向きと大きさ」を並べたものが**勾配** $\nabla\ell(w)$ である。重みの更新は，次の式で表される。

$$w \leftarrow w - \eta\,\nabla\ell(w; b) \tag{1}$$

η は**学習率**で，1 回にどれだけ動かすかの倍率である（本実験では 0.01）。b は**バッチ**で，データの中から取り出した一部（本実験では 32 枚）を指す。データ全体ではなく一部で勾配を計算し，少しずつ更新を繰り返す方法を，**確率的勾配降下法**（SGD）と呼ぶ。手元のデータを 1 周し終えることを 1 **エポック**という。

評価。 学習に使っていないデータ（**テストデータ**）でモデルに答えさせ，正解した割合を**精度**（accuracy）と呼ぶ。本実験のデータセット MNIST は，学習用 60,000 枚とテスト用 10,000 枚に分かれている。

2.2　連合学習とは

ふつうの機械学習では，データを 1 か所に集めて学習する。しかし，病院の診療記録やスマートフォンの入力履歴のように，持ち主の外に出したくないデータは集められない。**連合学習**（Federated Learning，FL）は，データを集めずに，**モデルのほうを各所へ送って学習させる**方法である。

登場するのは，まとめ役の**サーバ** 1 台と，データを持つ**クライアント**複数台である。次の一巡を**ラウンド**（通信ラウンド）と呼び，これを何度も繰り返す（図 1）。

1. サーバが，今のモデル（**グローバルモデル**）w^t を，全クライアントに配る。
2. 各クライアント k が，自分のデータだけを使って，式 (1) で重みを更新する（**ローカル学習**）。結果を w_k^{t+1} とする。
3. 各クライアントが，学習後の重み w_k^{t+1} をサーバに返す。**データそのものは送らない。**
4. サーバが，返ってきた重みを 1 つにまとめて（**集約**），新しいグローバルモデル w^{t+1} を作る。

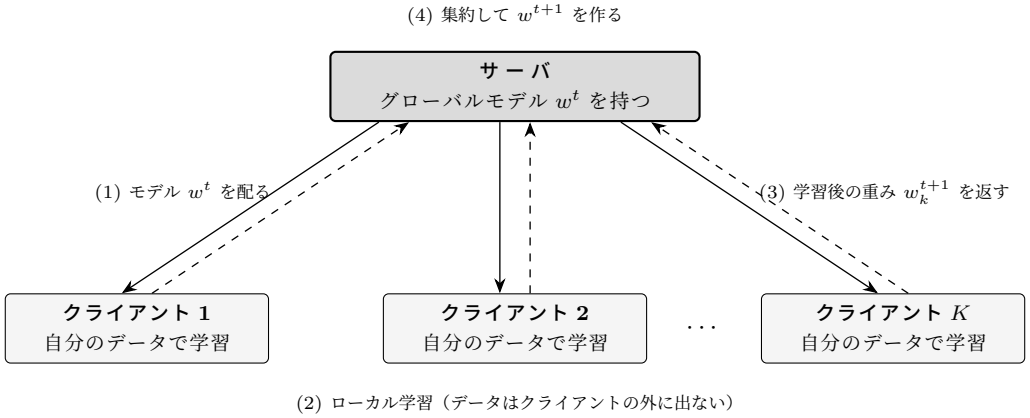


図 1　連合学習の 1 ラウンド。実線がサーバからクライアントへ，破線がクライアントからサーバへの通信。

本実験では，1 台のコンピュータの中で，サーバとクライアント 20 台を模擬的に動かしている（**シミュレーショ**

ン)。その仕組みを提供しているのが，連合学習用のライブラリ **Flower**（`flwr`）である。

2.3 FedAvg（集約の方法）

FedAvg（Federated Averaging）は，連合学習の一番基本的な集約の方法で，返ってきた重みを，**各クライアントのデータ数で重み付けして平均する**。クライアント k のデータ数を n_k，全体を $n = \sum_k n_k$ とすると，次の式になる。

$$w^{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{t+1} \tag{2}$$

重み付き平均の例。データ数が 100 枚のクライアント A と，300 枚のクライアント B がいて，ある重みの値が A は 2，B は 6 だったとする。

$$\frac{100 \times 2 + 300 \times 6}{100 + 300} = 5$$

単純な平均（4）より，データの多い B の値に近くなる。「データをたくさん持っているクライアントの学習結果ほど，重く扱う」という考え方である。

1 ラウンドの手順（本実験の設定）。

1. サーバが w^t を 20 台すべてに配る（毎ラウンド全員が参加する）。
2. 各クライアントが，手元のデータを 1 周（1 エポック）し，32 枚ずつのバッチで式 (1) を繰り返す。データが多いクライアントほど，更新の回数が多い。
3. サーバが式 (2) で平均する。

FedAvg の弱点。式 (2) は，届いた重みを**すべて信じて**平均に入れる。おかしな値を取り除く仕組みは無い。そのため，クライアントの中に，わざとおかしな重みを返す者（悪性クライアント）がいると，その影響がそのままグローバルモデルに入る。これに対処する方法は，大きく 2 つある。(a) 平均の代わりに中央値などを使い，外れた値の影響を受けにくくする（**頑健な集約**。Median，Krum など）。(b) おかしな更新を見つけて取り除く（**検知**）。本実験が調べているのは (b) の前半，つまり「そもそも見つけられるのか」である。

2.4 IID / Non-IID と Dirichlet 分割

IID と Non-IID。クライアントごとのデータの中身が，どれも似たような構成になっている状態を **IID**（独立同分布）という。たとえば，どのクライアントも 0〜9 の数字をほぼ同じ割合で持っている場合である。これに対して，クライアントによって中身が偏っている状態を **Non-IID** という。「あるクライアントは 3 と 7 ばかり，別のクライアントは 0 ばかり」という場合である。現実の連合学習は，ほぼ必ず Non-IID になる（人によって撮る写真も，打つ文章も違う）。

Non-IID だと，各クライアントは自分の偏ったデータに合うように重みを動かすので，**良性のクライアントどうしでも，学習結果がばらばらになる**。これが，本実験の中心にある問題である。良性どうしのばらつきが大きいと，悪性の「おかしさ」が，その中に埋もれてしまう。

Non-IID の種類。Non-IID には種類がある。

- **ラベル分布の偏り**：クライアントごとに，持っている数字（クラス）の比率が違う。
- **特徴分布の偏り**：同じ数字でも，見た目（筆跡など）がクライアントごとに違う。
- **データ数の偏り**：クライアントごとに，持っている枚数が違う。

本実験で扱うのは，このうち**ラベル分布の偏りだけ**である。枚数は全クライアントで同じにしている。特徴分布の偏りも与えていない。各クライアントには，同じ数字の画像の中から無作為に配っているので，どのクライアントの「7」も同じ集団から偶然選ばれたものであり，筆跡の傾向に系統的な違いはない。（書き手ごとにクライアントを分けたデータセット，たとえば FEMNIST を使うと，特徴分布の偏りが自然に生じる。）卒論で「特徴分布の偏りは**意図的には**与えていない」と書いているのは，偶然の小さな違いまでは否定できないためである。

Dirichlet 分割。本実験では，Non-IID の度合いを 1 つの数値 α で調整できるように，**ディリクレ分布**を使ってデータを分ける。設定の `data-split` が `"dirichlet_equal"` のときの分け方は，次のとおりである。

1. クライアント 1 台ごとに，「数字 $0, 1, \ldots, 9$ をどんな比率で持つか」の比率を，ディリクレ分布から乱数で 1 組引く。比率の合計は 1 である。
2. 全クライアントに，1 枚ずつ順番に画像を配る。配るたびに，そのクライアントの比率に従って，どの数字を配るかをくじで決める。ある数字の在庫がなくなったら，その数字はくじから外す。
3. これを 1,500 回くり返す。全クライアントが，ちょうど 1,500 枚ずつ持つことになる（設定の `samples-per-client`）。

比率の平均は，どの数字もデータ全体での割合（約 0.1）で同じである。違うのは，ばらつきの大きさで，標準偏差はおよそ $\sqrt{0.09\,/\,(\alpha+1)}$ になる。α が大きいほどばらつきが小さく（どのクライアントも全体と同じ比率に近い），α が小さいほどばらつきが大きい（偏る）。

表 1　α と偏りの関係（クライアント 20 台）。1 行目は理論値，残りは本実験の分割での実測。

	$\alpha = 1000$	$\alpha = 5$	$\alpha = 1$
比率の標準偏差（平均は約 0.1）	0.009	0.122	0.212
クライアントが持つ数字の種類（20 台の平均，`trial` 1）	10.0 種類	9.3 種類	5.8 種類
その中で一番多い数字が占める割合（同上）	12%	41%	66%
クライアントのデータ枚数	全員 1,500	全員 1,500	全員 1,500

注意：枚数をそろえている理由。 以前は，数字ごとに「どのクライアントに何割ずつ配るか」を引く方式（`data-split = "dirichlet"`）を使っていた。この方式では，α を小さくすると，「どの数字を持っているか」だけでなく，**「何枚持っているか」も大きく偏る**（$\alpha = 0.1$ では，最少 41 枚から最多 15,083 枚まで開く）。枚数が多いクライアントほど更新の回数が多く，更新が大きくなるので，検知のしやすさが変わっても，それが数字の偏りのせいなのか，枚数の偏りのせいなのかを区別できなかった。また，FedAvg は枚数で重み付けして平均するので，悪性クライアントの影響力が，台数の割合（0.2）からずれていた。枚数が全員同じなら，20 台の重みはどれも 1/20 で，悪性 4 台の影響力の合計はちょうど $4/20 = 0.2$ になる。枚数が不均一だと，悪性 4 台がたまたま枚数の少ないクライアントなら 0.2 より小さく，多いクライアントなら大きくなる（以前の方式の $\alpha = 0.1$ では 0.09〜0.16 だった）。すると，同じ「悪性割合 0.2」でも，攻撃の効き具合が α や `trial` によって変わってしまい，検知性能や精度の違いが，Non-IID のせいなのか，悪性の影響力の違いのせいなのかを区別できない。そこで，枚数を全クライアントで同じにし，数字の比率だけを α で変える方式に切り替えた。したがって，本実験の「Non-IID 度」は，ラベル分布の偏りだけを表す。

配る枚数を 1 台あたり 1,500 枚（合計 30,000 枚。全 60,000 枚の半分）にしているのは，全部を配りきる設定にすると，同じ数字を欲しがるクライアントが重なったときに在庫が足りなくなり，引いた比率どおりに配れなくなるためである。

3　攻撃と検知

3.1　悪性クライアントと，この実験の前提

連合学習では，サーバはクライアントのデータを見られない。見えるのは，返ってきた重みだけである。そのため，クライアントの一部が**悪性**で，学習を妨げるような重みをわざと返してきても，簡単には分からない。このように，重みを直接書き換えて学習を妨げる攻撃を，**モデルポイズニング**（model poisoning）と呼ぶ。

本実験の前提は，次のとおりである。

- 20 台のうち 4 台（割合 0.2）が悪性で，実験の最初から最後まで同じ 4 台である。
- 悪性クライアントどうしは協力できる（**結託**）。
- ALIE 攻撃では，悪性クライアントは，良性クライアントの更新を知っているものとする（攻撃者に有利な想定）。
- 悪性クライアントは，データ数は偽らずに申告する。
- サーバは正直で，どれが悪性かを知らない（実験では答え合わせのために正解を記録するが，検知器には渡さ

ない)。

3.2 更新ベクトル

クライアント k が，配られた重み w^t から，学習でどれだけ動かしたかを**更新ベクトル**と呼ぶ。

$$g_k = w_k^{t+1} - w^t \tag{3}$$

g_k は，重みの数（266,060 個）だけ数値が並んだ，1 本の長いベクトルである。検知器が見るのは，この g_k である（重みそのものではなく，動かした分を見る）。式 (2) は，$w^{t+1} = w^t + \sum_k (n_k/n)\, g_k$ と書き直せるので，FedAvg は「更新ベクトルの重み付き平均だけ，グローバルモデルを動かす」方法だともいえる。

ベクトルには，**大きさ**（長さ，ノルム）$\|g_k\|$ と**向き**がある。攻撃と検知の話は，ほとんどが「大きさ」と「向き」で説明できる。

高い次元での性質。 26 万次元のような高い次元では，互いに関係のない 2 本のベクトルは，ほぼ直角になる（実際に 266,060 次元で乱数のベクトルを 2 本作ると，なす角のコサインは 0.0003 だった）。そのため，2 本のベクトルが同じ方向を向いている，あるいは逆を向いている，というのは，偶然では起きない，意味のある情報になる。

3.3 符号反転攻撃（sign flip）

悪性クライアントは，ふつうに学習したあとで，更新ベクトルの符号を反転して送る。

$$g_k^{\mathrm{mal}} = -\, g_k \tag{4}$$

正直な更新は「損失が下がる向き」なので，その逆は「損失が上がる向き」である。平均に混ざると，学習を逆方向へ引っ張る。

この攻撃では，**大きさは変わらず**（$\|-g_k\| = \|g_k\|$），**向きだけが正反対**になる。そのため，大きさを見る検知器では見つけにくく，向きを見る検知器では見つけやすい。作り方が単純で，隠れる工夫をしていない攻撃である。

3.4 ALIE 攻撃（A Little Is Enough）

ALIE は，Baruch ら（2019）が提案した，**良性のばらつきの中に隠れる**攻撃である。考え方は，「良性クライアントどうしも，もともとある程度ばらついている。そのばらつきの範囲内で，少しだけ悪い方向にずらせば，見つからない」というものである。

重み 1 つ 1 つ（座標 j）について，良性クライアントの更新の平均 μ_j と標準偏差 σ_j を求め，悪性クライアント全員が，次の値を送る。

$$g_j^{\mathrm{mal}} = \mu_j - z\,\sigma_j \tag{5}$$

z は，ずらす強さを決める数値である。標準偏差は「ふつうのばらつきの幅」を表すので，z は「ふつうのばらつきの何倍だけ，平均からずらすか」を意味する。

例。 ある重みについて，良性の値の平均が 0.50，標準偏差が 0.10 で，$z = 1$ なら，悪性の値は 0.40 になる。良性の値は 0.40〜0.60 あたりに散らばっているので，0.40 は「端のほうにいる良性」に見える。

z の意味。 良性の値が正規分布に従うとすると，悪性の値 $\mu_j - z\sigma_j$ より外側（小さい側）にいる良性クライアントの割合は，z だけで決まる（表 2）。z が小さいほど，悪性の値は良性の集団の内側に入り，見つかりにくくなる。その代わり，モデルを動かす力は弱くなる。z が大きいほど，攻撃としては強くなるが，目立つ。スイープで z を 8 通り試しているのは，この兼ね合いを調べるためである。

表 2　z と，悪性の値より外側にいる良性クライアントの割合（正規分布を仮定した場合）

z	0.385	0.5	0.75	1.0
外側にいる良性の割合	35.0%	30.9%	22.7%	15.9%

0.385 は，原論文の式を 20 台・悪性 4 台に当てはめたときの z の上限の目安である。ただし，原論文の議論は「良性クライアントが IID で，値が正規分布に従う」という仮定に基づいており，Non-IID でも同じ意味を持つとは

限らない。本実験では，同じ作り方の攻撃を，いろいろな Non-IID 度に当てはめたらどうなるかを，実験的に調べている。

符号反転との違い。

- 悪性クライアントは**全員まったく同じ値**を送る。
- 良性の更新を知らないと作れない。そのため，コードではサーバ側で差し替えている（クライアントは互いの値を見られないので，攻撃者が全員の値を知っている状況を，サーバ側で模擬している）。

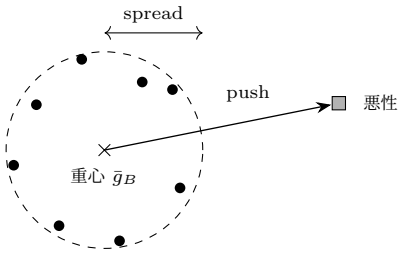
3.5 広がり（spread）と押し出し（push）

「悪性は，良性の集団の内側にいるのか，外側にいるのか」を数値にしたものが，本実験の中心になる指標である。良性クライアントの集合を B，悪性クライアントの集合を M とし，良性の更新の平均（**重心**）を $\bar{g}_B$ とする。

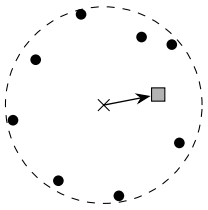
$$\text{spread} = \frac{1}{|B|} \sum_{i \in B} \|g_i - \bar{g}_B\| \qquad \text{（良性が，重心からどれだけ散らばっているか）} \tag{6}$$

$$\text{push} = \frac{1}{|M|} \sum_{i \in M} \|g_i - \bar{g}_B\| \qquad \text{（悪性が，良性の重心からどれだけ離れているか）} \tag{7}$$

$$\text{push_ratio} = \text{push} \,/\, \text{spread} \tag{8}$$



push_ratio > 1：悪性は集団の外側（見つけやすい）



push_ratio < 1：悪性は集団の内側（紛れる）

図 2　spread と push の考え方（実際は 26 万次元だが，2 次元で描いた）。黒い点が良性，灰色の四角が悪性，破線の円が良性の広がり。

ALIE では，push_ratio はほぼ z になる。 式 (5) から，悪性と良性の重心との距離は $\|g^{\text{mal}} - \bar{g}_B\| = z\,\|\sigma\|$ である（σ は σ_j を並べたベクトル）。一方，良性の「距離の 2 乗の平均」の平方根は，ちょうど $\|\sigma\|$ になる（標準偏差の定義そのもの）。距離の平均（spread）は，これより大きくはならないので，

$$\text{push_ratio} = \frac{z\,\|\sigma\|}{\text{spread}} \;\geq\; z \tag{9}$$

が成り立ち，高い次元では，ほぼ等号になる。つまり ALIE では，z **を決めると push_ratio がほぼ決まる**。しかも，α を変えても push_ratio は変わらない。ALIE では，push_ratio は測って分かる量ではなく，z で決めた量である。そこで，push_ratio と検知性能の関係は符号反転だけで調べ，ALIE は z を攻撃の強さの軸として別に調べる。スイープでは，z を 1 より小さい値から大きい値まで 8 通りに振る。

中心仮説。 本実験の中心にある仮説は，「push_ratio が 1 を下回ると（悪性が良性の広がりの内側に入ると），検知が崩れる」というものである。Non-IID が強いほど良性の広がり（spread）が大きくなるので，同じ強さの攻撃でも，内側に入りやすくなる。上に述べた理由で，この仮説を push_ratio で確かめるのは符号反転だけとする。ALIE は，push_ratio をほぼ一定にしたまま Non-IID の強さだけを変える条件にあたる。同じ z で α によって検知性能が変われば，それは push_ratio では説明できない違いである。

3.6 検知という問題

サーバの手元には，20 本の更新ベクトルがある。その中から，悪性のものを見つけたい。本実験の検知器は，各クライアントに「**悪性らしさのスコア**」を付ける。スコアが大きいほど，悪性らしいと考える。

「スコアがいくつ以上なら悪性と判定するか」という線引き（**しきい値**）は，決めていない。スコアで順位を付けるだけにして，その順位がどれだけ正解と合っているかを測る（§4.1）。しきい値の決め方によらずに，検知器そのものの良し悪しを比べるためである。

以下の 4 つの検知器は，どれも「多数派と違うものを疑う」という考え方に立っている。違うのは，**何が違うことを見るか**である。

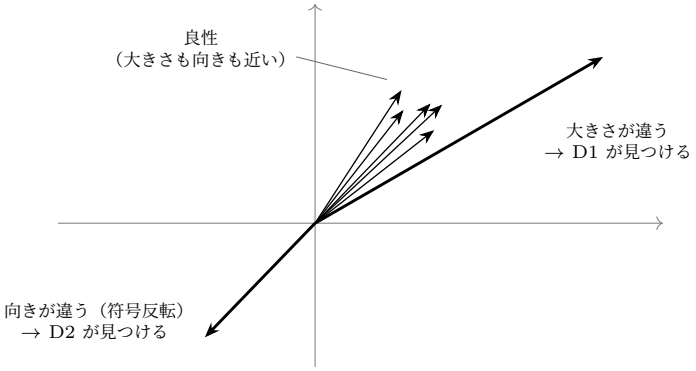


図 3　大きさと向き。矢印 1 本が，クライアント 1 台の更新ベクトルを表す（2 次元で描いた概念図）。

3.7　検知器 D1：大きさを見る（ノルム偏差）

各クライアントの更新の大きさ $\|g_i\|$ を測り，全クライアントの**中央値**からどれだけ離れているかをスコアにする。

$$s_i^{\mathrm{D1}} = \Big|\, \|g_i\| - \operatorname*{median}_j \|g_j\| \,\Big| \tag{10}$$

- **考え方**：ほかと比べて，極端に大きい（または小さい）更新は怪しい。
- **中央値を使う理由**：平均は，極端な値が 1 つあるだけで大きく動く。中央値（大きさの順で真ん中の値）は，半分近くが極端な値にならない限り動かないので，悪性に引っ張られにくい。
- **得意**：更新を何倍にも大きくする攻撃。
- **苦手**：大きさが良性と変わらない攻撃。符号反転は大きさが同じなので，原理的に見分けられない。ALIE も，良性の範囲内の大きさになる。また，Non-IID でデータ量が偏ると，良性どうしでも大きさが大きく違うので，悪性かどうかではなくデータ量に反応してしまう。

同点について。 クライアントの台数が偶数のとき，中央値は真ん中の 2 台の平均になる。すると，その 2 台は中央値からちょうど同じ距離にあり，スコアは数式の上で必ず等しくなる（大きさが 2，4，10，20 なら，中央値は 7 で，4 と 10 のスコアはどちらも 3）。計算の丸め誤差でこれがずれて，意味のない順位が付くのを防ぐため，コードでは，ごく近い値を同じ値にそろえている。

3.8　検知器 D2：向きを見る（コサイン距離）

まず，代表的な向き r（**参照方向**）を決める。重み 1 つ 1 つについて，クライアント間の中央値を取ったベクトルである。次に，各クライアントの更新と r のなす角を，**コサイン**で測る。

$$\cos(g_i, r) = \frac{g_i \cdot r}{\|g_i\|\,\|r\|}, \qquad s_i^{\mathrm{D2}} = 1 - \cos(g_i, r) \tag{11}$$

$g_i \cdot r$ は内積（同じ位置の値どうしを掛けて，全部足したもの）である。コサインは，同じ向きなら 1，直角なら 0，正反対なら -1 になる。長さで割っているので，大きさには左右されない。スコアは，同じ向きで 0，直角で 1，正反対で 2 になる。

- **考え方**：みんなが向いている方向と違う方向を向いている更新は怪しい。
- **得意**：符号反転。正反対を向くので，スコアが最大になる。
- **苦手**：向きを大きく変えない攻撃。

注意：ALIE では，スコアが逆に出る。 ALIE に対しては，D2 は悪性を「見つけられない」のではなく，**悪性のほうに，良性より低いスコア（より良性らしい）を付ける**傾向がある。これまでの実行では，ROC-AUC がほぼ 0 になっていた。考えられる理由は，次のとおりである。良性クライアントの更新は，それぞれ固有のずれ（自分のデータの偏りによるもの）を持っていて，そのずれは互いにばらばらの方向を向いている。一方，ALIE の更新は良性の平均をもとに作るので，この固有のずれを持たない。しかも，悪性 4 台のずれ（$-z\sigma$）は全員同じなので，中央値で作る参照方向 r が，そちらに少し引き寄せられる。その結果，悪性は誰よりも参照方向に近くなる。いわば，「典型的すぎる」のである。この説明は推測を含むので，新しい結果で確かめる必要がある。

3.9 検知器 D3：まとめて見る（k-means）

大きさと向きの両方を使い，クライアントを 2 つのグループに分けて，多数派から遠いものを疑う。

1. 各クライアントを，4 つの数値（**特徴量**）で表す。更新の大きさ $\|g_i\|$，D2 のスコア，第 1 主成分の成分，第 2 主成分の成分（主成分は D4 の項で説明する）。
2. 4 つの数値を**標準化**する。それぞれについて，(値 − 平均) ÷ 標準偏差 を計算し，平均 0・ばらつき 1 にそろえる。大きさと向きでは値の桁が違うので，そろえないと，桁の大きいものだけで結果が決まってしまう。
3. **k-means 法**で，2 つのグループに分ける。
4. 人数の多いグループを「良性」とみなし，その中心 c までの距離をスコアにする：$s_i^{\mathrm{D3}} = \|z_i - c\|$（$z_i$ は標準化した特徴量）。

k-means 法。 データを，似た者どうしのグループに分ける方法である。(1) グループの中心を仮に決める，(2) 各データを近いほうの中心のグループに入れる，(3) 各グループの平均を新しい中心にする，という手順の (2) と (3) を，変わらなくなるまで繰り返す。最初の中心の置き方によって，違う分け方に落ち着くことがある（置き方が悪いと，もっと良い分け方にたどり着けない）。ふつうは最初の中心を乱数で何度か変えて試すが，それでは実行のたびに結果が変わりうる。本実験では，最初の中心を全部の組み合わせ（20 台なら 190 通り）で試し，グループが一番よくまとまった分け方を採用している。乱数を使わないので，同じ入力なら必ず同じ結果になる。

- **考え方**：悪性は少数派で，多数派とは違うまとまりを作るはずである。
- **得意**：D1 か D2 の片方でしか見えない攻撃にも対応できる。
- **苦手**：悪性が多数派になると，前提が成り立たない。また，良性どうしのばらつきが大きく，グループがはっきり分かれない場合は，分け方が悪性・良性の区別と関係のないものになる。

3.10 検知器 D4：一番ばらついている方向を見る（スペクトル）

主成分分析（PCA）。 26 万次元のデータを，そのまま眺めることはできない。そこで，「クライアントによる違いが一番はっきり出る方向」を探す。

教室の生徒の身長と体重を，横軸と縦軸に取って点を打つと，点は右上がりの細長い雲になる。その雲が一番長く伸びている向き（「体が大きい／小さい」にあたる向き）を**第 1 主成分**と呼ぶ。それと直角で，次に広がっている向き（「太め／細め」にあたる向き）が**第 2 主成分**である。本実験では，身長・体重の 2 個が，26 万個の重みに増えているだけである。

手順。

1. 各クライアントの更新から，全クライアントの平均を引く：$\tilde{g}_i = g_i - \bar{g}$（**中心化**。位置ではなく，中心からのずれを見る）。
2. 中心化した更新を行に並べた表（行列）の，第 1 主成分の方向 v（長さ 1 のベクトル）を求める。計算には**特異値分解**（SVD）を使う。
3. 各クライアントのずれを v に射影し（v 方向の成分を取り出し），その 2 乗をスコアにする。

$$s_i^{\mathrm{D4}} = (\tilde{g}_i \cdot v)^2 \tag{12}$$

2 乗するのは，v のプラス側に離れていても，マイナス側に離れていても，同じように「離れている」と数えるためである。

- **考え方**：悪性クライアントがまとまって同じ方向にずれていると，「クライアント間の違いが一番大きい方向」が，ちょうど「悪性と良性を分ける方向」になる。その方向に大きく突き出しているものが悪性である。バックドア攻撃の検知で提案された，スペクトル・シグネチャ（Tran ら，2018）の考え方に基づく。
- **得意**：悪性が全員同じ更新を送る攻撃（ALIE）や，全員が同じ向きにずれる攻撃。
- **苦手**：良性どうしの違いのほうが大きい場合。このとき，第 1 主成分は良性どうしの違いを表す方向になり，悪性は目立たない。Non-IID が強いと，まさにこれが起きうる。

分布構造の診断指標。 D4 がうまく働いたかどうかを考える材料として，本実験では次の指標も記録している。

- **第 1 主成分の寄与率**（`pc1_ratio`）：ばらつき全体のうち，第 1 主成分の方向が何割を占めるか。1 に近ければ細長い形，小さければ丸い形に散らばっている。
- **悪性の方向と第 1 主成分の重なり**（`mal_cos_pc1`）：悪性のずれている方向と，第 1 主成分の方向の，コサインの絶対値。1 に近ければ，D4 が見ている方向が悪性の方向と一致している。
- **悪性の値の順位**（`mal_rank_mean`）：重み 1 つ 1 つについて，悪性の値以下である良性が何割いるかを求め，全部の重みで平均したもの。

これらは正解を使って計算するので，検知器ではなく，考察のための指標である。

3.11 4 つの検知器のまとめ

検知器	見るもの	得意	苦手
D1	更新の大きさ	極端に大きい・小さい更新	大きさが良性と同じ攻撃。データ量の偏り。
D2	更新の向き	向きを逆にする攻撃（符号反転）	向きを変えない攻撃。ALIE には逆向きに反応する。
D3	大きさ・向き・主成分をまとめて	片方でしか見えない攻撃	悪性が多数派の場合。グループが分かれない場合。
D4	一番ばらつきの大きい方向	悪性がそろって同じ方向にずれる攻撃	良性どうしのばらつきが大きい場合。

4 評価とスイープ

4.1 ROC-AUC

検知器の成績は，**ROC-AUC**（以下 AUC）で測る。意味は，次のとおりである。

> 悪性クライアントを 1 台，良性クライアントを 1 台，でたらめに選んだとき，**悪性のほうがスコアが高くなっている確率**。

AUC	意味
1.0	どの組でも，悪性のほうがスコアが高い。完全に見分けられる。
0.5	半々。でたらめに順位を付けたのと同じで，見分けられていない。
0.0	どの組でも，悪性のほうがスコアが低い。悪性のほうが「良性らしい」と判定されている。**見分けられていないのではなく，逆向きにはっきり分かれている**（スコアの向きを逆にすれば，完全に見分けられる）。

名前の由来。 しきい値を決めると，「悪性のうち，正しく悪性と判定できた割合」（真陽性率）と，「良性のうち，誤って悪性と判定した割合」（偽陽性率）が決まる。しきい値をいろいろに変えて，この 2 つを縦軸と横軸に取って描い

た曲線を ROC 曲線といい，その下の面積（Area Under the Curve）が AUC である。この面積が，上に書いた確率と等しくなることが知られている。しきい値を 1 つに決めなくてよいので，検知器どうしを公平に比べられる。

計算の例。 クライアントが 5 台で，A と C が悪性だとする。

クライアント	A	B	C	D	E
正解	悪性	良性	悪性	良性	良性
スコア	0.9	0.2	0.3	0.4	0.1

「悪性 1 台と良性 1 台」の組は，$2 \times 3 = 6$ 組ある。A（0.9）は，B・D・E の 3 台すべてより高い。C（0.3）は，B と E より高いが，D（0.4）より低い。悪性のほうが高い組は 6 組中 5 組なので，$\mathrm{AUC} = 5/6 \approx 0.83$ である。スコアが同じ値（同点）の組は，0.5 組として数える。

攻撃なしの条件。 悪性が 0 台だと，「悪性と良性の組」が作れないので，AUC は計算できない（NaN と記録される）。これは正常である。

4.2 そのほかの指標

- **上位 f 台での再現率・適合率**（`recall_at_f`，`precision_at_f`）：「スコアの高い順に，上位 f 台を悪性と判定する」と決めたときの成績。f は実際の悪性の台数（本実験では 4）。再現率は，本当の悪性のうち，上位 f 台に入っていた割合。適合率は，上位 f 台のうち，本当に悪性だった割合。f が悪性の台数と同じなので，この 2 つは同じ値になる。上の例（$f = 2$）では，上位 2 台は A と D なので，どちらも 0.5 である。
- **平均適合率**（`ap`）：スコアの高い順に見ていき，悪性が現れるたびに「ここまでの中で悪性は何割か」を記録して，平均したもの。上の例では，A の位置で 1/1，C の位置で 2/3 なので，$(1 + 2/3)/2 \approx 0.83$ である。
- **精度**（accuracy）：グローバルモデルが，テストデータ 10,000 枚のうち何割に正しく答えたか。検知の成績ではなく，学習がどこまで進んだか，攻撃でどれだけ妨げられたかを見るための値である。

4.3 スイープとは

条件と実行。 1 回の連合学習を，最初のラウンドから最後のラウンドまで通して行うことを，1 回の**実行**と呼ぶ。実行のときに決めておく設定の組み合わせ（Non-IID 度は $\alpha = 1$，攻撃は ALIE，$z = 0.5$，…）が，1 つの**条件**である。

スイープ。 知りたいのは，「Non-IID 度を変えると，検知はどう変わるか」「攻撃の強さを変えるとどうか」といった，設定と結果の関係である。そのためには，設定を少しずつ変えながら，何度も実行する必要がある。このように，**設定をいくつかの値に振って，その組み合わせをすべて実行すること**を，**スイープ**（sweep）と呼ぶ。振る設定のことを**軸**という。

本実験の軸は，Non-IID 度（α），攻撃の種類，ALIE の強さ（z），繰り返しの番号（trial）である。

表 3　本実験のスイープ（各条件が 10 回の実行，計 300 回）。ALIE の z は 0.385，0.5，0.75，1.0，1.25，1.5，2.0，3.0 の 8 通り

	攻撃なし	符号反転	ALIE（z 8 通り，各 10 回）	計
$\alpha = 1000$（ほぼ IID）	10	10	80	100
$\alpha = 5$（中程度）	10	10	80	100
$\alpha = 1$（強い偏り）	10	10	80	100

繰り返し（trial）と乱数の種。 実験には，乱数で決まる部分がある（データの分け方，モデルの最初の重み，どのクライアントが悪性か，学習時のデータの順番）。1 回だけの実行では，結果が設定のせいなのか，たまたまの巡り合わせなのかを区別できない。そこで，同じ条件を，乱数だけを変えて 10 回繰り返し，平均とばらつき（標準偏差）を見る。この繰り返しの番号が **trial** である。

乱数は，**種**（seed）と呼ぶ数値を決めると，毎回同じ並びで出てくる。本実験では trial 番号を種にしているので，同じ trial 番号なら何度実行しても同じ結果になり（**再現性**），trial 番号を変えると別の結果になる。

攻撃なしの条件を入れる理由。悪性がいないので AUC は計算できないが，「良性だけのときの広がり（spread）」と「攻撃がないときの精度」の基準が得られる。攻撃ありの結果は，これと比べて読む。

第 II 部
コードの全体像

5　全体像

連合学習，攻撃，検知の考え方は第 I 部で説明した。ここでは，それがコードの中でどのファイルに書かれ，どうつながっているかを示す。

5.1　ファイルの配置と，実験の流れ

図 4 に，ファイルの配置と，実験が進む順番を示す。設定を書く場所は `pyproject.toml` の 1 か所だけで，人が直接実行するのは `run_experiments.py` と `analyze_detection.py` の 2 つである。

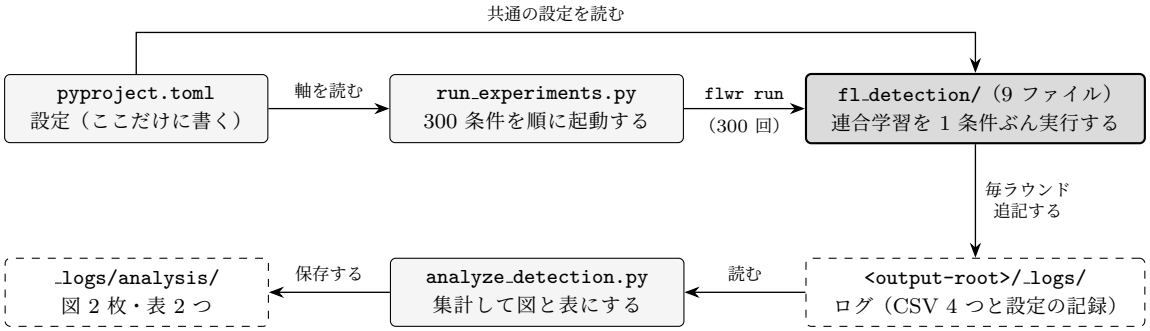


図 4　ファイルの配置と実験の流れ。実線の箱がプログラムと設定，破線の箱が出力されるデータ。濃い箱が，連合学習の実行中に動くコード。

役割は，次の 3 つに分かれている。

場所	役割	補足
`fl_detection/`	連合学習の実行中に動く	研究の中身（学習，攻撃，検知，集約，記録）はすべてここにある。
`run_experiments.py`	実行を外から繰り返す	条件を変えながら `flwr run` を 300 回呼ぶだけ。研究の中身は持たない。
`analyze_detection.py`	実行後の結果を読む	ログを読んで平均を取り，図と表にするだけ。実験コードには依存しない。

5.2　`fl_detection/` の中のつながり

図 5 に，9 つのファイルのあいだの「誰が誰を使っているか」を示す。矢印は「使う側 → 使われる側」である。

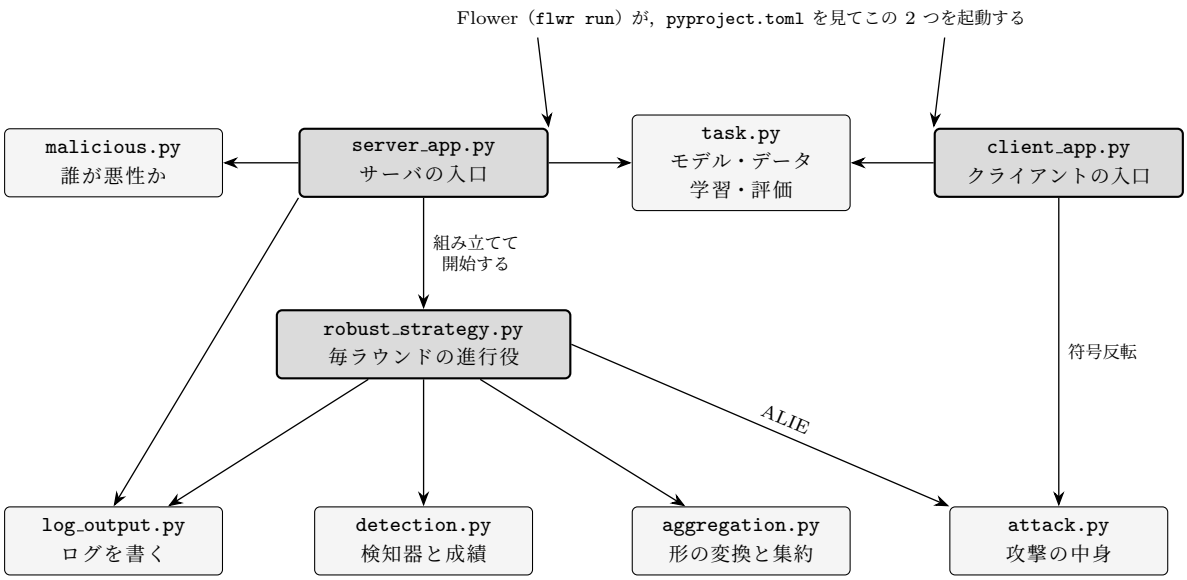


図 5　fl_detection/ の 9 ファイルの関係（__init__.py は目印だけのファイルなので省略）。濃い箱は Flower から呼ばれる側で，薄い箱は値を受け取って値を返すだけの部品。client_app.py は画面表示のために log_output.py も使うが，図では省いた。

読み取ってほしいのは，次の 3 点である。

- **入口は 2 つ。** Flower が起動するのは server_app.py と client_app.py だけで，ほかのファイルは，この 2 つ（と robust_strategy.py）から呼ばれて初めて動く。
- **研究の中心は** robust_strategy.py**。**攻撃の差し込み，検知スコアの計算，記録，集約を，毎ラウンド順番に呼び出している。server_app.py に攻撃や検知が出てこないのは，この中に入っているためである。
- **攻撃の入り口は 2 か所。** 符号反転はクライアントが自分で行い，ALIE はサーバ側で差し替える（§26）。

5.3　1 ラウンドのあいだに起きること

図 6 に，1 ラウンドの処理を，サーバ側とクライアント側に分けて示す。各ステップの下に，その処理が書かれているファイルと関数を記した。

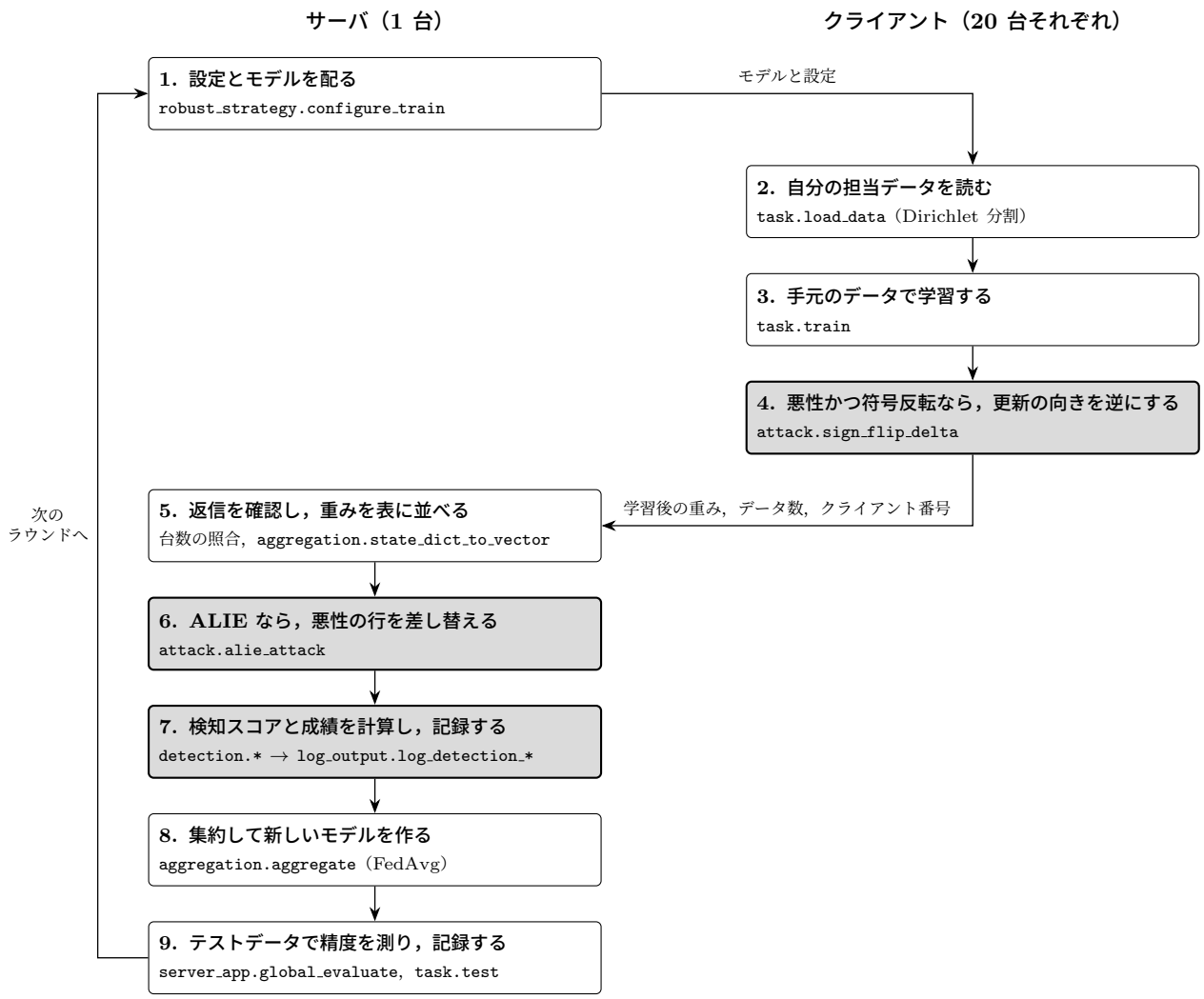


図 6　1 ラウンドの流れ。濃い箱が，攻撃と検知に関わるステップ。ステップ 7 がこの研究の中心で，ここで記録した値が実験の結果になる。ステップ 7 は記録するだけで，ステップ 8 の集約には影響しない。

6　各ファイルの説明

ファイルごとに，役割，主な関数，読むときの要点をまとめる。各行の意味は，ファイルの中にコメントとして書いてある（「【この部分の説明】」などの見出しが付いている）。

6.1　読む順番の目安

1. `server_app.py`：実験全体がどう始まり，どう終わるか。
2. `client_app.py`：クライアント 1 台が何をするか。
3. `robust_strategy.py`：毎ラウンド，サーバ側で何が起きるか（研究の中心）。
4. `attack.py`，`detection.py`：攻撃と検知の中身。
5. `task.py`，`aggregation.py`，`malicious.py`，`log_output.py`：必要になったときに。

6.2　`fl_detection/server_app.py`（サーバの入口）

役割

サーバ側の本体。`flwr run` で最初に実行される `main()` が入っている。自分ではほとんど計算をせず，次の順で全体を指揮する。

流れ

(1) 設定を読む → (2) 出力先とログの保存先を決める → (3) 悪性クライアントを決める → (4) 設定を表示し，`run_config.txt` に記録する → (5) 乱数の種を固定する → (6) 最初のモデルを作る → (7) 戦略（`RobustFedAvg`）

を組み立てる → (8) 毎ラウンドの採点のしかた（global_evaluate）を定義する → (9) 連合学習を開始する → (10) 最終モデルと最終結果を保存する。

要点

実行時間のほとんどは，(9) の strategy.start(...) の 1 行で費やされる。ここで全ラウンドが回り，終わるまで先へ進まない。それより上は準備，下は後片付けである。global_evaluate は，ここでは定義するだけで，実行するのは Flower（毎ラウンド 1 回）である。

6.3 fl_detection/client_app.py（クライアントの入口）

役割

クライアント 1 台ぶんの処理。同じコードが，クライアントの台数ぶん呼び出される。自分が何番のクライアントかは，実行時に Flower から受け取る。

主な関数

train()：サーバからの「学習して」という依頼に応える（毎ラウンド，全クライアント）。

evaluate()：「採点して」という依頼に応える（毎ラウンド，一部のクライアント。結果は画面に表示するだけ）。

train() の流れ

設定を取り出す → 届いた重みでモデルを作る → 自分の担当データを読む → 学習する → 学習後の重みを取り出す → 悪性なら攻撃する → サーバへ返す。

要点

攻撃は，更新量 Δ = 学習後の重み − 学習前の重み に対して行う。悪性かつ攻撃が符号反転のときだけ，Δ の符号を反転する。ALIE のときは，悪性クライアントもここでは何もせず，正直な結果を返す。

6.4 fl_detection/robust_strategy.py（毎ラウンドの進行役）

役割

「毎ラウンド何をするか」を担当するクラス RobustFedAvg。Flower 標準の FedAvg を継承し，3 つのメソッドだけを書き直している。

主なメソッド

configure_train：学習の依頼を出す前。設定を配り，配る前のグローバルモデルを控える。

configure_evaluate：評価の依頼を出す前。設定を配る。

aggregate_train：学習結果が届いた後。**攻撃の差し込み，検知，記録，集約**を行う。

aggregate_train の流れ

(1) 返信を確認し（台数の照合），番号順に並べる → (2) 重みをベクトルに並べ直して表にする → (3) どの行が悪性かを特定する → (4) ALIE なら悪性の行を差し替える → (5) 良性の広がりと悪性の押し出し具合を測る → (6) 検知スコアを計算して記録する → (7) 成績（ROC-AUC など）を計算して記録する → (8) 集約する。

要点

これらのメソッドを呼ぶのは Flower で，呼び出しはどのファイルにも書かれていない。扱うデータは，行がクライアント（20 行），列がモデルの重み 1 つ 1 つ（266,060 列）の表である。正解（どれが悪性か）は成績を付けるためだけに使い，検知器には渡していない。

6.5 fl_detection/attack.py（攻撃の中身）

役割

攻撃の計算だけを書いたファイル。「いつ，誰に対して攻撃するか」は，呼び出す側が決める。

主な関数

sign_flip_delta：更新量の符号を反転する。client_app.py から呼ばれる。

alie_attack：良性の平均 μ と標準偏差 σ から $\mu - z\sigma$ を作り，悪性の行に入れる。robust_strategy.py から呼ばれる。

alie_z_max：原論文の式による z の目安を計算する。実験中は呼ばれない（テスト用）。

要点

符号反転は 1 台ぶんの「層ごとの辞書」を，ALIE は全クライアントぶんの「表」を扱う。ALIE がサーバ側で動くのは，ほかのクライアントの値を見ないと作れない攻撃だからである。

6.6 `fl_detection/detection.py`（検知器と成績）

役割

検知に関する計算だけを書いたファイル。3 つの部分に分かれている。

1) **検知器**

`all_detector_scores`：4 つの検知器（D1–D4）のスコアをまとめて計算する。どの検知器も「スコアが大きいほど悪性らしい」向きにそろえてある。正解は使わない。

2) **分布構造の診断指標**

`structure_diagnostics`：良性の更新がどんな形で散らばり，悪性がその中のどこにいるかを測る。考察用で，正解を使う。検知器ではない。

3) **検知性能指標**

`evaluate_scores`：スコアと正解から，ROC-AUC，平均適合率，上位 f 台での再現率と適合率を計算する。

要点

しきい値（どの値から悪性と判定するか）は決めていない。スコアで順位を付けるだけで，その順位がどれだけ正解と合っているかを ROC-AUC で測る。主成分分析や ROC-AUC の計算など，数学として難しい箇所の補足を，ファイルの末尾にまとめてある。

6.7 `fl_detection/task.py`（モデル・データ・学習）

役割

連合学習の仕組みとは関係のない，ふつうの機械学習の部品。

主な関数

`Net`：手書き数字の画像を受け取り，0〜9 のどれかを答える CNN（重みは 266,060 個）。

`load_data`：クライアント 1 台が使うデータを用意する。中で Dirichlet 分割を行う。

`load_centralized_dataset`：サーバが採点に使うテストデータ（10,000 枚）を用意する。

`train` / `test`：モデルを学習させる / 採点する。

要点

Non-IID を作っているのは，この中の `_build_dirichlet_equal_partitions` である。クライアント 1 台ごとに，「数字 0〜9 をどんな比率で持つか」を乱数で決め，その比率で全員に同じ枚数（1,500 枚）を配る。α を小さくすると持っている数字が偏るが，**枚数は全クライアントで同じ**である（§2.4）。以前の方式（枚数も偏る）は `_build_dirichlet_partitions` として残してある。各クライアントは，担当ぶんの 8 割（1,200 枚）を学習に，2 割を自分の評価に使う。

6.8 `fl_detection/aggregation.py`（形の変換と集約）

役割

モデルの重みを，「層ごとの辞書」と「1 本の長いベクトル」のあいだで変換する処理と，集約（FedAvg）。

主な関数

`state_dict_to_vector` / `vector_to_state_dict`：形の変換。

`stack_updates`：クライアントごとのベクトルを，1 つの表に積む。

`aggregate`：手法の名前を受け取り，その手法で集約する（現在は `fedavg` だけ）。

要点

平均を取る，更新の大きさや向きを測る，といった計算は，層の区別なく全部の重みを同列に扱いたい。そのため，集約だけでなく ALIE 攻撃と検知も，このベクトルの形で計算している。FedAvg は，各クライアントのデータ数で重み付けした平均である。

6.9 fl_detection/malicious.py（誰が悪性か）

役割

どのクライアントを悪性にするかを決める。実験の最初に 1 回だけ使われ，決まった番号は最後まで変わらない。

主な関数

generate_malicious_ids：台数，割合，乱数の種から，悪性にするクライアントの番号を選ぶ。

要点

台数 × 割合が整数にならない設定は，エラーになる。スイープでは乱数の種に trial を渡すので，trial ごとに悪性になるクライアントが変わる（20 台・割合 0.2 の場合，trial 1 では 8, 9, 14, 19 番）。

6.10 fl_detection/log_output.py（ログを書く）

役割

記録をファイルに書き出す。計算はしない。渡された値を，決まった列の順番で，ファイルの末尾に書き足すだけである。

主な関数

configure_log_dir：書き出し先のフォルダを切り替える。

log_run_config：設定の全項目を run_config.txt に書く。

log_detection_csv / log_detection_metrics_csv：検知のログを書く。

log_results_csv / log_summary_csv：精度のログを書く。

要点

どのファイルも上書きではなく追記で，300 条件が同じファイルに順番に書き足される。どの行がどの条件のものかは，各行に入っている条件の列で見分ける。

6.11 run_experiments.py（スイープの起動）

役割

条件を変えながら，flwr run を順番に呼ぶ。人が手でコマンドを 300 回打つ代わりをしているだけで，研究の中身は持たない。

流れ

pyproject.toml を読む → 条件の一覧を作る → Flower のコマンドとクライアントの台数を確かめる → 条件を 1 つずつ実行する → 完了・スキップ・失敗の件数を表示する。

要点

条件ごとに，最後に作られる final_model.pt があるかどうかで，成功か失敗かを判断する（flwr run は，中でエラーが起きても「成功」と返すことがあるため）。このファイルがある条件は飛ばすので，途中で止めても，もう一度実行すれば続きから回る。失敗した条件は <output-root>/failures.csv に記録し，次の条件へ進む。

6.12 analyze_detection.py（結果の分析）

役割

実験が終わったあとに，detection_metrics.csv を読んで，図と表を作る。成績は実験中に計算済みなので，ここでは平均を取って並べ替え，図に描くだけである。

主な関数

load_metrics：ログを読む。回し直した条件があれば，最後の実行だけを残す。

plot_auc_by_round / plot_pushratio_vs_auc / plot_auc_by_z：図を描く。

pushratio_corr_table / final_auc_table / structure_table：表を作る。

要点

実験コード（fl_detection/，Flower，pyproject.toml）には依存しない。このファイル 1 つとログのフォルダがあれば，どこでも分析できる。

第 III 部
使い方

7　環境の構築

7.1　必要なもの

- Python 3.10 以上（動作を確認した環境は Python 3.12）。
- **GPU は必須ではない。**再スイープ（300 条件）は CPU で実行し，約 62 時間かかった。GPU を使える環境なら，クライアントの学習の部分を速くできる（§7.3）。
- インターネット接続（最初の 1 回だけ。パッケージと MNIST を取得する）。

必要なパッケージは `pyproject.toml` の `dependencies` に書いてあり，下の手順でまとめて入る。動作を確認した版を表 4 に示す。

表 4　動作を確認したパッケージの版

パッケージ	版	用途
`flwr`（Flower）	1.33.0	連合学習の実行基盤
`torch` / `torchvision`	2.8.0 / 0.23.0	モデルと学習
`flwr-datasets` / `datasets`	（自動で入る）/ 3.1.0	MNIST の取得
`numpy`	2.5.2	検知・攻撃・集約の計算
`pandas` / `matplotlib`	3.0.5 / 3.11.1	分析（`analyze_detection.py`）

`numpy`，`pandas`，`matplotlib` は，`flwr-datasets` が必要とするパッケージとして一緒に入る。scikit-learn は使っていない（検知器と成績の計算は，`fl_detection/detection.py` に numpy だけで書いてある）。

7.2　手順

プロジェクトのフォルダ（`pyproject.toml` がある場所）で，次を実行する。

Windows（本番の環境）。

```
python -m venv .venv
.venv\Scripts\activate
pip install -e .
```

Mac / Linux。

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

以下の表と，本書のコマンド例は Mac / Linux の書き方で示す。Windows では，`source .venv/bin/activate` を `.venv\Scripts\activate` に読み替える（PowerShell で実行できない場合は `.venv\Scripts\Activate.ps1`）。

コマンド	意味
`python3 -m venv .venv`	このプロジェクト専用の Python 環境（仮想環境）を，`.venv` というフォルダに作る。ほかのプロジェクトとパッケージの版がぶつからないようにするためのもの。最初の 1 回だけ行う。
`source .venv/bin/activate`	仮想環境を有効にする。以後，このターミナルで打つ `python` や `flwr` は，`.venv` の中のものになる。**ターミナルを開き直すたびに必要。**
`pip install -e .`	`pyproject.toml` に書かれたパッケージを，仮想環境に入れる。最初の 1 回だけ行う。

入ったかどうかは，次で確かめられる。テストがすべて「合格」と表示されればよい。

```
flwr --version
python test_detection.py
```

7.3 GPU を使うための設定

GPU で学習させるには，次の 3 つがそろっている必要がある。

	必要なこと	確かめ方・設定する場所
1	PyTorch が GPU 対応の版である	`python -c "import torch; print(torch.cuda.is_available())"` が `True` になる。
2	計算に使う装置の設定	`pyproject.toml` の `[tool.flwr.app.config]` の `device`。`"auto"` なら GPU があれば GPU を使う。`"cuda"` にすると，GPU が使えないときにエラーで止まる。
3	クライアントへの GPU の割り当て	`pyproject.toml` の `[tool.sweep]` の `client-num-gpus`（0 より大きい値。現在は 0.2）。

1. PyTorch の版。Windows では，`pip install -e .` で入る PyTorch は CPU 専用の版になることがある。上の確認で `False` と表示された場合は，GPU（CUDA）対応の版を入れ直す。入れ直すコマンドは，PyTorch の公式サイト（`pytorch.org` の「Get Started」）で，自分の環境に合うものを選んで表示させる。版は `torch` 2.8.0 / `torchvision` 0.23.0 に合わせる。次は，CUDA 12.6 向けの例である（CUDA の版は環境によって違うので，このまま使えるとは限らない）。

```
pip install torch==2.8.0 torchvision==0.23.0 --index-url https://download.pytorch.org/whl/cu126
```

3. クライアントへの割り当てが必要な理由。Flower は，クライアントを 1 台ずつ別のプロセスで動かす。NVIDIA の GPU は，サーバからは常に見えるが，クライアントからは，割り当てたときだけ見える。割り当てを忘れると，「GPU で回しているつもりが，クライアントは CPU で学習している」という状態が，エラーなしで起きる。`client-num-gpus` は，クライアント 1 台に GPU の何割を割り当てるかを表す。

- 0.2 なら，1 枚の GPU を同時に最大 5 台で分け合う（20 台は 5 台ずつ順に学習する）。
- 値を小さくすると同時に動く台数が増えて速くなるが，GPU のメモリを多く使う。メモリ不足（`CUDA out of memory`）になる場合は，値を大きくする。
- GPU の無いマシンでは，必ず 0.0 にする。0 より大きいままだと，クライアントが起動できない。

設定の食い違いは，回す前に止まる。`run_experiments.py` は，実行の最初に GPU の設定を確かめ，次の場合は 1 条件も回さずに止まる。(a) `client-num-gpus` が 0 より大きいのに，そのマシンで NVIDIA の GPU が使えない。

(b) NVIDIA の GPU が使えるのに `client-num-gpus` が 0.0 で，`device` が `"cpu"` でない。

実際に GPU が使われたかの確かめ方。 実行の最初に画面に表示される `device : cuda` の行と，`run_config.txt` の `device（effective）`の行で，サーバが使った装置が分かる。クライアントが GPU を使っているかは，実行中に別のターミナルで `nvidia-smi` を実行し，Python のプロセスが GPU を使っているかを見る。

速さについて。 GPU で速くなるのは，クライアントの学習の計算の部分だけである。サーバ側の処理（検知スコアの計算，採点に使うテストデータの読み込みと前処理）と，条件ごとの起動（約 1 分）は，GPU を使っても短くならない。参考として，Apple Silicon の Mac で測ったところ，20 台で 1 ラウンドあたり CPU が約 8.2 秒，内蔵 GPU が約 7.1 秒だった。Windows の NVIDIA GPU での速さは，まだ測っていない。

7.4 クライアントの台数の設定

「実際に何台のクライアントを起動するか」は，`pyproject.toml` ではなく，Flower 側の設定（`num-supernodes`）で決まる。この台数と `pyproject.toml` の `num-clients` が一致していないと，実験は最初のラウンドでエラーになって止まる（§27）。

- `run_experiments.py` でスイープを回す場合は，実行の最初に，スクリプトが Flower 側の台数を `num-clients` に合わせる。手で設定する必要はない。
- 手で `flwr run` を実行する場合は，自分で合わせる（§9.3）。

7.5 初回実行時の注意（MNIST のダウンロード）

初回の実行で，HuggingFace Hub から MNIST（約 11 MB）が自動でダウンロードされる。2 回目以降は，手元に保存されたものが使われる。`Warning: You are sending unauthenticated requests to the HF Hub` と表示されても，動作に支障はない。

8 設定（pyproject.toml）

8.1 設定はここだけに書く

実験の設定値は，`pyproject.toml` の 2 つの欄だけに書く。Python のファイルには設定値を書かない。

欄	内容
`[tool.flwr.app.config]`	全実行で共通の値。手で `flwr run .` を実行したときは，この値で動く。
`[tool.sweep]`	スイープで変えながら回す軸と，出力先のフォルダ名。 `run_experiments.py` だけが読む。

スイープでは，`[tool.sweep]` の軸に対応する項目だけが，条件ごとに上書きされる。それ以外の項目（台数，ラウンド数，学習率など）は，全条件で `[tool.flwr.app.config]` の値が使われる。

8.2 共通の設定

表 5 [tool.flwr.app.config] の項目。「上書き」は，スイープで条件ごとに上書きされる項目。

項目	値	上書き	意味
num-server-rounds	50		通信ラウンド数。
num-clients	20		クライアントの総数。実際に起動する台数と一致させる。
local-epochs	2		各クライアントが，1 ラウンドで手元のデータを何周するか。
learning-rate	0.01		学習率（1 回の更新で重みを動かす倍率）。
batch-size	32		学習で一度に使うデータの枚数。
device	"auto"		計算に使う装置。auto / cuda（NVIDIA の GPU）/ mps（Mac の GPU）/ cpu。auto は，使える中で一番速いものを選ぶ。
fraction-evaluate	0.2		クライアント側の評価に参加する割合（結果は画面に表示するだけ）。
aggregation-method	"fedavg"	○	集約手法。現在使えるのは fedavg だけ。
data-split	"dirichlet_equal"		データの分割方式。dirichlet_equal（枚数は全員同じ）と dirichlet（以前の方式。枚数も偏る）が使える。
samples-per-client	1500		クライアント 1 台に配る枚数（dirichlet_equal のときだけ使う）。
dirichlet-alpha	5.0	○	Dirichlet の濃度。大きいほど IID に近く，小さいほど偏る。
attack-type	"none"	○	攻撃。none / sign_flip / alie。
alie-z	1.0	○	ALIE の強さ z（ALIE のときだけ上書き）。
malicious-ratio	0.2	○	悪性クライアントの割合。num-clients との積が整数になる値にする。攻撃なしのときは，この値に関係なく悪性 0 台として扱われる。
malicious-seed	1	○	悪性クライアントを選ぶ乱数の種（スイープでは trial と同じ値）。
trial	1	○	繰り返しの番号。乱数の種として，データ分割・モデルの初期値・学習時の並べ替えに使う。
output-dir	"."	○	出力先。final_model.pt はここに，ログはその 1 つ上のフォルダの _logs/ に出る。

8.3 スイープの軸

表 6 [tool.sweep] の項目

項目	値	意味
output-root	"experiments_detection_v3"	出力先の親フォルダ。
client-num-gpus	0.2	クライアント 1 台に割り当てる NVIDIA GPU の割合（§7.3）。GPU の無いマシンでは 0.0。
aggregation-methods	["fedavg"]	集約手法（1 通り）。
dirichlet-alphas	[1000.0, 5.0, 1.0]	Non-IID 度（3 通り）。
attack-types	["none", "sign_flip", "alie"]	攻撃（3 種類）。
alie-zs	[0.385, 0.5, 0.75, 1.0, 1.25, 1.5, 2.0, 3.0]	ALIE の z（8 通り。ALIE のときだけ使う）。
trials	[1, 2, ..., 10]	繰り返し（10 回）。

条件の数は，$1 \times 3 \times$ (攻撃なし 1 + 符号反転 1 + ALIE 8) $\times 10 = 300$ である。

決まり。コードを変えてログの列が変わったら，output-root の名前を変える（例：_v3）。ログは同じファイルに追記されていくので，列を変えた後に同じフォルダへ書き足すと，見出しと合わない行が混ざってしまう。

9 実行手順

以下は，仮想環境を有効にした状態（source .venv/bin/activate）で行う。

9.1 手順 1：テスト

コードに手を入れたら，まずテストを回す。それぞれ「すべてのテストに合格しました。」と表示されればよい。

```
python test_detection.py
python test_attack.py
python test_aggregation.py
python test_malicious.py
```

9.2 手順 2：条件の確認

実行はせず，条件の一覧と，実際に使われるコマンドを表示する。pyproject.toml を変えたら，まずこれで意図どおりかを確かめる。

```
python run_experiments.py --dry-run
```

先頭に，条件の数（300）と，共通の設定（台数，ラウンド数，学習率など）が表示される。

9.3 手順 3：動作確認（1 条件だけ短く回す）

スイープの前に，1 条件だけ短く回して，ログが出るかを確かめる（1 行で入力する）。

```
flwr run . local-simulation --stream --run-config
    'num-server-rounds=2 num-clients=10 attack-type="alie" alie-z=0.5 output-dir="_smoke/run1"'
```

- 指定していない項目は，pyproject.toml の値で動く。
- num-clients は，実際に起動するクライアントの台数と一致させる。local-simulation の台数は ~/.flwr/config.toml の num-supernodes で決まり，既定は 10 である。
- 完了すると，_smoke/_logs/detection_updates.csv に，1 ラウンドあたり num-clients 行が記録される。
- 確認用の出力（_smoke/）は消してよい。
- このコマンドは Mac / Linux の書き方である。Windows の PowerShell やコマンドプロンプトでは，引用符の書き方が違うため，このままでは動かないことがある。Windows では，手順 2 の --dry-run で条件を確かめたうえで，手順 4 のスイープを始め，最初の 1 条件が完了するのを見届ける方法が確実である。

9.4 手順 4：スイープ

```
python run_experiments.py
```

- 300 条件を順に実行する。所要時間は環境によって大きく変わるので，最初の 1 条件にかかった時間を 300 倍して見積もる。参考：以前の 54 条件のスイープは，CPU で約 12 時間（1 条件あたり約 13 分）かかった。再スイープの 300 条件は，CPU で約 62 時間（1 条件あたり約 11 分，回し直しを含む）かかった。Windows の NVIDIA GPU での所要時間は，まだ測っていない。
- 途中で止めても，もう一度同じコマンドを実行すれば，未完了の条件から続く。
- 失敗した条件は experiments_detection_v3/failures.csv に記録され，スイープは止まらない。
- **最初の 1 条件が完了するまで（数分）は，様子を見ること。**台数の食い違いなどの設定の誤りは，最初の条件の最初のラウンドで止まる形で現れる。GPU で回す場合は，このあいだに nvidia-smi で，GPU が使われていることも確かめる。

9.5 手順 5：分析

```
python analyze_detection.py experiments_detection_v3/_logs
```

引数は，detection_metrics.csv が入っているフォルダである（省略できない）。実行すると，読み込んだファイルの場所，行数，実行の数，ラウンドの範囲が表示されるので，意図したデータかを確かめる。図と表は，指定したフォルダの中の analysis/ に出力される（--outdir で変更できる）。最後の何ラウンドを平均するかは --last-k で指定する（既定は 5）。

最初に見る図。 検知は「そのラウンドの更新ベクトルの集まりを見分けられるか」という問題なので，モデルが最終的な精度に達している必要はない。ただし，AUC がラウンドをまたいで大きく暴れると，結果の解釈が難しくなる。分析したら，まず analysis/detection_auc_by_round.png を見て，AUC の推移が解釈できる範囲に収まっているかを確かめる。学習の進み具合は results.csv の精度で確かめられる。学習率やラウンド数を変える場合は，pyproject.toml の learning-rate と num-server-rounds を書き換え，output-root の名前も変えてから回し直す。

10 出力ファイル

スイープの出力は，すべて <output-root>/（既定では experiments_detection_v3/）の下にできる。

表 7　出力されるファイル。行数は，300 条件・50 ラウンド・20 台の場合。

ファイル	行数の目安	内容
<条件名>/final_model.pt	300 個	最終のグローバルモデル。完了したかどうかの目印にもなる。
_logs/run_config.txt	300 かたまり	実行ごとに，実際に使われた設定の全項目と，悪性クライアントの番号。
_logs/detection_updates.csv	300,000 行	生の記録。1 クライアント × 1 ラウンドで 1 行。
_logs/detection_metrics.csv	60,000 行	成績の記録。1 検知器 × 1 ラウンドで 1 行。分析の入力。
_logs/results.csv	15,000 行	ラウンドごとの精度と損失（サーバがテストデータで測った値）。
_logs/summary.csv	300 行	最終ラウンドの精度と損失。
failures.csv	（失敗時のみ）	失敗した条件の名前，コマンド，出力の末尾。
_logs/analysis/	4 ファイル	分析の出力（図 2 枚，表 2 つ）。

どの CSV にも，実験条件を表す列（aggregation_method，data_split，alpha，attack_type，alie_z，malicious_ratio，trial）と，実行を見分ける run_id が入っている。

10.1 detection_updates.csv（1 クライアント 1 行）

集約の直前（攻撃の適用後）の各クライアントの更新について記録する。検知の入力は更新ベクトル $g_i = w_i - w_{\text{global}}$（届いた重みから，配った重みを引いたもの）である。

表 8　detection_updates.csv の主な列

列	内容
round	通信ラウンドの番号（1 から始まる）。
client_pos / partition_id	表の中の行の位置 / クライアントの番号（番号順に並べているので，同じ値になる）。
is_malicious	正解（1 が悪性）。**成績を付けるためだけに使う。**検知器には渡していない。
delta_norm	更新ベクトルの長さ $\|g_i\|_2$。
d1_norm 〜 d4_spectral	検知器 D1–D4 のスコア。大きいほど悪性らしい。
benign_spread	そのラウンドの，良性クライアントの広がり（同じラウンドの行では同じ値）。
push_ratio	そのラウンドの，悪性の押し出し距離 ÷ 良性の広がり（悪性 0 台のときは空）。
pc1_ratio など 6 列	分布構造の診断指標（同じラウンドの行では同じ値）。

10.2　detection_metrics.csv（1 検知器 1 行）

実験中に robust_strategy.py が計算して記録する。1 行が「ある実行の，あるラウンドの，ある検知器の成績」である。

表 9　detection_metrics.csv の主な列

列	内容
detector	検知器の名前（d1_norm / d2_cosine / d3_kmeans / d4_spectral）。
auc	ROC-AUC。悪性 1 台と良性 1 台を選んだとき，悪性のスコアのほうが高い確率。
ap	平均適合率（average precision）。
recall_at_f / precision_at_f	スコアの上位 f 台を悪性と判定したときの再現率 / 適合率（f は実際の悪性の台数）。
n_clients / n_malicious	そのラウンドのクライアントの台数 / 悪性の台数。
benign_spread, push_ratio など	detection_updates.csv と同じ指標。
mean_dist_unfiltered / mean_dist_filtered	**参考の値（記録するだけ）。**検知器が疑った上位 f 台を仮に取り除いたとき，残りの平均が，良性の重心からどれだけ離れているか。unfiltered は何も取り除かない場合，filtered はその検知器のスコア上位 f 台を取り除いた場合。小さいほど良性だけの平均に近い。

参考の値について。mean_dist_* の 2 列は，研究計画書に当初あった「防御効用」にあたる値である。本研究を「検知できるかどうか」に絞ったため，計画書の評価指標からは外し，卒業論文では扱わない。それでも記録しているのは，計算に更新ベクトルそのものが必要で，スイープの後からは計算できないためである（増える実行時間は，スイープ全体で約 20 秒）。使う場合は，ここでの平均が単純な平均であり，実際の集約（データ数で重み付けする FedAvg）とは一致しないことに注意する。analyze_detection.py は，この 2 列を使わない。

ROC-AUC の読み方。1.0 は「悪性のほうが常にスコアが高い」（完全に見分けられる），0.5 は「でたらめと同じ」，0.0 は「悪性のほうが常にスコアが低い」（悪性のほうが良性らしいと判定されている）である。0.0 に近い値は，「見分けられていない」のではなく，「逆向きにはっきり分かれている」ことを意味する。攻撃なしの条件は悪性が 0 台なので，auc は NaN（計算できない）になる。これは正常である。

10.3 分析の出力（analysis/）

ファイル	内容
detection_auc_by_round.png	検知器（行）× 攻撃（列）ごとに，AUC のラウンド推移を α 別に描いた図。線が trial の平均，帯が標準偏差。
detection_pushratio_vs_auc.png	符号反転だけを対象にした，横軸 push_ratio，縦軸 AUC の散布図（中心仮説の主図）。1 点が 1 実行の 1 ラウンド。右下の数値は順位相関係数。
detection_pushratio_corr_table.csv	符号反転での push_ratio と AUC の順位相関係数（全体と α ごと）。α ごとの値が小さければ，両者の関係は α の違いで説明できてしまう。
detection_auc_by_z.png	ALIE だけを対象にした，横軸 z，縦軸 AUC の図。最後の数ラウンドの平均を trial ごとに求め，その平均と標準偏差を α 別に描く。
detection_final_auc_table.csv	最後の数ラウンドの AUC（α× 攻撃 × 検知器）。trial ごとに平均してから，trial のあいだの平均（_mean）と標準偏差（_std）を出す。
detection_structure_table.csv	最後の数ラウンドの，広がり・push_ratio・分布構造の診断指標と，AUC の対応表。
accuracy_by_round.png	テスト精度のラウンド推移（α ごと。攻撃なし・符号反転・ALIE の $z=1$ と $z=3$）。results.csv があるときだけ作る。
accuracy_table.csv	最終ラウンドの精度（trial 間の平均・標準偏差・最小）と，最後の数ラウンドでの精度の上がり幅（学習が収束しているかの目安）。

第 IV 部
卒論の主張と根拠

11 この部の使い方

この部は，卒論（Memo/soturon2.tex）を吟味するための手引きである。卒論の主張を 1 つずつ取り上げ，次の 4 点をまとめる。

1. **何を主張しているか。**
2. **根拠はどこにあるか。**どの表・図か，どのログのどの列か。
3. **その数値をどう計算したか。**自分で計算し直せるように書く（§24）。
4. **どこまで確かか。**下の 4 段階で示す。

表 10　主張の確かさの 4 段階。

段階	意味
【観測】	ログの数値そのもの。集計の方法が正しければ，疑う余地はない。
【予測で確認】	実験の**前に**予測を書き，結果がそのとおりになったもの。後付けの説明より強い。
【解釈】	観測を説明する考え方。筋は通っているが，別の説明がありうる。
【推測】	確かめていない説明。論文では「可能性がある」と書くべきもの。

使うデータ。 すべて experiments_detection_v3/_logs/ にある（再スイープ，300 回の実行）。

- detection_metrics.csv：ラウンドごと・検知器ごとの AUC と，spread・push_ratio・分布構造の指標。
- detection_updates.csv：ラウンドごと・クライアントごとのノルムと検知スコア（.gz を展開したもの）。
- results.csv：ラウンドごとのテスト精度。 summary.csv：最終ラウンドの精度。
- run_config.txt：実行ごとの設定。300 回すべてで，計画どおりの設定（エポック 2，学習率 0.01 など）だったことを確認済み。
- analysis/：analyze_detection.py が作った図と表。卒論の表の多くは，ここの CSV から作った。

途中で失敗して回し直した実行の行もログに残っているが，集計では条件ごとに最後の実行だけを使う（keep_last_runs）。また，1 回の実行（$\alpha = 1000$，ALIE $z = 3$，trial 9）だけ，detection_updates.csv の第 48 ラウンドの行が欠けている。AUC は detection_metrics.csv から取るので影響はない。

卒論に書いていないこと。 以前の 54 回の実行（データ数も偏る分け方，$z \leq 1$，各 3 試行）は，卒論には書いていない。実験の設計を変えた理由（データ数を揃えた，$z > 1$ を足した，試行を 10 回にした）と，その実行で分かったことは，本書の §30，この部の各節の「読むときの注意」，および研究計画書（sotsuron_plan2.tex）に残してある。卒論の「事前の予測が当たった」という主張（§17）は，この以前の実行を見て立てた解釈から，再スイープの前に計画書に書いた予測にもとづく。審査で「その予測はどこから来たのか」と聞かれたら，以前の実行の結果から立てた解釈だと答えればよい。

ラウンド推移も主張の一部である。 卒論は，AUC を「各ラウンドで測れる指標」として使い，学習の段階によって検知性能がどう変わるかを追っている（関連研究の章の「本研究の特徴」の 1 つ目）。「収束していなくても測れる」という便利さではなく，「途中の変化が見える」ことを利点として書いている。実際，D2 は ALIE（$z = 1.5$，2）を序盤には見つけていたのに後半には逆転し，D1 は符号反転を序盤はほとんど見つけられず後半に見つけるようになった（detection_auc_by_round.png）。最終 5 ラウンドの値だけを見ると，これらは見えない。$\alpha = 1$ が収束していないこと（§14）も，この立場から「学習の段階による違い」として扱う。

卒論の節	本書の節
実験結果「学習の収束と攻撃が精度に与える影響」	§14（精度と収束），§22（ALIE はほぼ無害）
実験結果「検知性能の全体像」「AUC のラウンド推移」	§13（全体像）
考察「検知器間の違い」	§15（D2），§16（D3），§20（符号反転と D1）
考察「Non-IID 度が検知性能に与える影響」	§18（D4 と分布の形），§19（push_ratio）
考察「攻撃のステルス性と検知破綻の境界」	§17（高次元の殻）
考察「中心仮説の評価」	§19，§23（予測と結果）
実験結果「正常更新の分布構造」	§18，§21（裾の重さ）

卒論の節との対応。

12 図と表の読み方

卒論の第 8 章（実験結果）の図と表を読むための手引きである。図表ごとに，**何を描いているか**，**まずどこを見るか**，**この実験での読み取り例**，**読み間違えやすい点**の 4 つをまとめる。ファイルはすべて experiments_detection_v3/_logs/analysis/ にある（卒論の Memo/fig/ にはその写しを置いている）。

すべての図表に共通する約束。

- **AUC の目盛り**：1 は完全に見分けられる，0.5 は見分けられない，0 は**逆向きに**完全に分かれている（§4.1）。図の点線（AUC = 0.5）より上か下かで，まず「正しい向きか，逆向きか」を判断する。

- **平均とばらつき**：特に断りがなければ，値は 10 回の試行（trial）の平均，帯や棒は試行間の標準偏差である。帯や棒が長いのは，「試行によって結果が割れている」ことを意味する。
- α **の色**：赤の実線（○）が $\alpha = 1000$（ほぼ IID），青の破線（□）が $\alpha = 5$，緑の一点鎖線（△）が $\alpha = 1$（強い Non-IID）。白黒で印刷しても，線の種類と印で区別できるようにしてある。
- **「最終 5 ラウンド」**：第 46〜50 ラウンドのこと。条件を 1 つの数値にまとめるときに使う。

12.1 精度の推移（accuracy_by_round.png）と精度の表（accuracy_table.csv）

何を描いているか。 横に 3 枚，左から $\alpha = 1000, 5, 1$。横軸がラウンド，縦軸がテスト精度（%，60% 以上だけを表示）。線は攻撃なし（黒の実線），符号反転（赤の破線），ALIE の $z = 1$（青）と $z = 3$（緑）。表には，最終ラウンドの精度（平均・標準偏差・最小）と，最後の 5 ラウンドでの上がり幅（gain_last_5）がある。

まずどこを見るか。

1. 黒の線（攻撃なし）が右端で平らになっているか → 学習が収束しているか。表の上がり幅で数値を確かめる。
2. 黒の線と他の線の差 → 攻撃がどれだけ精度を下げたか。

この実験での読み取り例。

- $\alpha = 1000, 5$ は上がり幅が 0.3 ポイント未満で，ほぼ収束。$\alpha = 1$ は 0.5〜0.9 ポイントで，まだ上がっている。
- 符号反転（赤）は $\alpha = 1$ で 76% 前後に落ち，帯も広い（試行によって 63〜85% まで割れる）。
- ALIE は，$\alpha = 1000, 5$ では黒の線とほとんど重なる（ほぼ無害）。$\alpha = 1$ だけ，$z = 3$ で黒より下にはっきり離れる。

読み間違えやすい点。 $\alpha = 1$ の符号反転の上がり幅は 0.05 と小さいが，これは「収束した」のではなく，攻撃で学習が止まっているだけである。上がり幅は，精度そのものと合わせて読む。

12.2 最終 AUC の表（卒論の表 tab:final-auc，detection_final_auc_table.csv）

何を描いているか。 行が攻撃（ALIE の 8 つの z と符号反転），列が α ごとの D1〜D4。値は最終 5 ラウンドの AUC。求め方は，まず試行ごとに 5 ラウンドを平均し，その 10 個の値の平均を取る。卒論の表では，試行間の標準偏差が 0.10 以上のときだけ「±」で書いている。CSV には，全部の値の平均（_mean）と標準偏差（_std）がある。

まずどこを見るか。

1. 0 に近い値（逆向き）と，0.5 付近の値（見分けられない）を探す。この 2 つは意味が違う。
2. 同じ行（同じ攻撃）で，α によって値が大きく変わる検知器を探す → Non-IID が効いている場所。
3.「±」が付いた値 → 試行によって結果が割れている場所。平均だけで結論を出さない。

この実験での読み取り例。 D2 の列は，ALIE の $z \leq 1.5$ ですべて 0.00（逆向き）。D1 の列は $z = 1.25$ の行だけが 0.09〜0.16 と低い。D4 の列は，$z = 0.75$ の行で α によって 1.00 / 0.39 / 0.16 と大きく変わる。

読み間違えやすい点。

- 0.00 を「全く検知できない」と読まない。スコアの向きを逆にすれば完全に見つかる状態である。
- 1 回のラウンドの AUC は，悪性 4 台 × 良性 16 台 = 64 組の比較で決まるので，1/64 刻みの粗い値になる。平均すれば細かい値になるが，0.01 程度の差には意味を持たせない。

12.3 z と AUC の図（detection_auc_by_z.png）

何を描いているか。 ALIE だけを対象に，横に 4 枚（D1〜D4）。横軸が z，縦軸が最終 5 ラウンドの AUC。α ごとに 1 本の線で，棒は試行間の標準偏差。最終 AUC の表の ALIE の部分を，検知器ごとに線でつないだものと考えてよい。

まずどこを見るか。

1. 線の形：右上がりか，谷があるか，ある z で急に跳ぶか。

2. 3 本の線が離れている $z \to$ その z では Non-IID が検知のしやすさを変えている。ALIE では push_ratio が z でほぼ決まるので，ここでの差は push_ratio では説明できない差である。

この実験での読み取り例。

- D1：$z = 1.25$ に深い谷があり，その前後では高い（§17「高次元の殻」）。
- D2：$z = 1.5$ まで 0 に張り付き，$z = 2$ で赤だけが上がり，$z = 3$ で全部上がる。
- D3：どの z でも上の方で平ら。
- D4：低い位置から，ある z で 1.0 に跳ぶ。跳ぶ位置は赤（$\alpha = 1000$）が一番左で，Non-IID が強いほど右にずれる（§18）。

読み間違えやすい点。 横軸の z は等間隔ではない（0.385, 0.5, 0.75, 1, 1.25, 1.5, 2, 3）。点と点の間の線は「その間を測っていない」ので，跳ぶ位置は「2 つの点の間のどこか」としか言えない。

12.4 AUC のラウンド推移（detection_auc_by_round.png）

何を描いているか。 格子状の小さなグラフ。行が検知器（上から D1〜D4），列が攻撃（ALIE の各 z と，右端が符号反転）。1 枚の中で，横軸がラウンド，縦軸がその回の AUC。線が試行の平均，薄い帯が標準偏差。analysis/ の図は ALIE の 8 水準をすべて並べた 9 列だが，卒論の図（Memo/fig/detection_auc_by_round_v3_sel.png）は $z = 0.75, 1.25, 1.5, 2$ と符号反転の 5 列に絞った版である。

まずどこを見るか。

1. 線が右端（終盤）でどこにいるか → 最終 AUC の表の値と同じ。
2. 序盤と終盤で値が大きく違う線 → 学習の段階で検知のしやすさが変わる場所。最終 5 ラウンドの値だけでは見えない。
3. 帯の幅 → 試行による割れ方。

この実験での読み取り例。

- D2 の ALIE $z = 1.5$：最初の数ラウンドだけ高く，10 ラウンド以内に 0 へ落ちる。$z = 2$ でも，青（$\alpha = 5$）はゆっくり，緑（$\alpha = 1$）は 10 ラウンドほどで 0 近くまで落ちる。赤（$\alpha = 1000$）は高いまま。
- D1 の符号反転：0.5 付近から始まり，ラウンドとともに 0.9 前後まで上がる（§20）。
- D4 の符号反転：$\alpha = 1$（緑）だけが，ラウンドとともに 1.0 から 0.86 くらいまで下がる。

読み間違えやすい点。 第 1 ラウンドの値は，初期のモデルからの 1 回分の更新なので，不安定になりやすい。序盤の 1〜2 ラウンドの急な上下だけで結論を出さない。卒論では，第 1〜5，6〜15，16〜30，31〜50 ラウンドの区間ごとに平均した値で傾向を述べている（§24）。

12.5 push_ratio と AUC の散布図（detection_pushratio_vs_auc.png）と相関の表

何を描いているか。 符号反転だけを対象に，2 × 2 の 4 枚（D1〜D4）。横軸が push_ratio（**対数目盛り**），縦軸が AUC。1 つの点が「1 回の実行の 1 ラウンド」なので，各 α で 10 試行 × 50 ラウンド = 500 点ある。右下の枠の数値は順位相関係数 ρ（Spearman）。1 行目の all が全部の点，その下が α ごと。同じ数値は detection_pushratio_corr_table.csv にもある。

まずどこを見るか。

1. 中心仮説が正しければ，点は「左（push_ratio が小さい）ほど下（AUC が低い）」に並び，ρ は正になるはずである。そうなっているかを見る。
2. 色ごとの点の固まりの位置 → α によって push_ratio がどう違うか。
3. all の ρ と，α ごとの ρ の違い。

この実験での読み取り例。 D2・D3 は push_ratio によらず，ほぼ全部の点が 1.0 に並ぶ。ρ は全体でも α ごとでも弱いか負で，仮説の形にはなっていない。例外は $\alpha = 1$ の D1（$\rho = 0.69$）だけである。

読み間違えやすい点。

- **all の ρ だけで判断しない。**符号反転の push_ratio は主に α で決まるので，all の相関は「α が違うと AUC が違う」ことを言い換えているだけかもしれない。α ごとの ρ でも同じ向きなら，push_ratio そのものが効いていると言いやすい。
- n/a は，その α で AUC がほぼ常に 1.0 で，順位が付けられず計算できないことを表す（相関がないのではない）。
- 序盤のラウンドでは，$\alpha = 1000$ の点が push_ratio 10 前後まで右に伸びる。散布図には全ラウンドの点が入っているので，序盤と終盤が混ざっている。

12.6 spread・push_ratio の表と，分布構造の表（detection_structure_table.csv）

何を描いているか。条件ごとに，最終 5 ラウンドで平均した「説明する側の指標」を並べた表。卒論では，spread と push_ratio の表と，分布構造の表に分けて載せている。

表 11　分布構造の表の列と，値の読み方。

列（卒論の表での名前）	何を測っているか	値が大きいと
benign_spread（spread）	良性が重心からどれだけ散らばっているか（平均距離）	良性どうしのばらつきが大きい
push_ratio	悪性の重心からの距離 ÷ spread	悪性が良性の殻より外にいる（1 を境に内・外）
pc1_ratio（寄与率・正常）	良性のばらつきのうち，一番大きい 1 方向が占める割合	良性が特定の方向に細長く広がっている（丸くない）
pc1_ratio_all（寄与率・全更新）	同じ量を，悪性も含めた全員で求めたもの（D4 が見ている行列）	全員のばらつきが 1 方向に集中している
mal_cos_pc1（$\lvert\cos\theta\rvert$・正常）	悪性のずれの方向と，良性の第 1 主成分の重なり（0〜1）	悪性が良性の主なばらつきの方向にずれている
mal_cos_pc1_all（$\lvert\cos\theta\rvert$・全更新）	悪性のずれの方向と，全員の第 1 主成分の重なり	悪性のずれが「一番目立つ方向」になっている（D4 が見つけやすい）
mal_rank_mean（順位・平均）	各座標で「悪性の値以下の良性が何割か」の平均	悪性が良性の分布の中ほどにいる（正規分布なら $\Phi(-z)$）
mal_rank_std（順位・SD）	上の割合の，座標によるばらつき	座標によって，悪性の位置がまちまち

まずどこを見るか。

1. 攻撃なしの行の spread と寄与率を α で比べる → Non-IID が良性の「大きさ」と「形」のどちらを変えたか。
2. 同じ z の行を α で比べ，$\lvert\cos\theta\rvert$（全更新）と D4 の AUC が一緒に動いているかを見る。

この実験での読み取り例。

- 攻撃なしで，spread は 0.143 / 0.143 / 0.155 とほぼ同じなのに，寄与率は 0.086 / 0.122 / 0.193 と 2 倍以上違う（§19，§18）。
- ALIE の $z = 0.75$ で，$\lvert\cos\theta\rvert$（全更新）0.920 / 0.305 / 0.138 と D4 の AUC 0.998 / 0.385 / 0.161 が同じ順に並ぶ。

読み間違えやすい点。

- spread と push_ratio 以外の列は，正解ラベル（誰が悪性か）を使って計算した「考察のための量」である。検知器はこれらを知らない。
- $\lvert\cos\theta\rvert$（全更新）は D4 の判定のしかたそのものに近いので，D4 の AUC と連動するのはある意味当然である。「原因」ではなく「D4 が失敗する様子の言い換え」として読む（§18）。
- 攻撃なしの行には悪性がいないので，悪性に関する列は空欄（NaN）になる。

13 結果の全体像

細かい数値に入る前に，検知器ごとの結果を 1 枚にまとめる（表 12）。数値は最終 5 ラウンドの AUC（`trial` ごとに平均してから 10 試行の平均。`analysis/detection_final_auc_table.csv`）。3 つの数値は，$\alpha = 1000/5/1$ の順である。

表 12　検知器 × 攻撃の AUC のまとめ（$\alpha = 1000/5/1$）。0.5 は見分けられない，0 に近いのは逆向きに分かれている（§4.1）。

検知器	符号反転	ALIE $z = 0.385$	ALIE $z = 1.25$	ALIE $z = 3$
D1 ノルム	0.97 / 0.94 / 0.88	1.00 / 1.00 / 0.97	**0.16 / 0.09 / 0.14**	1.00 / 1.00 / 1.00
D2 コサイン	1.00 / 1.00 / 1.00	**0.00 / 0.00 / 0.00**	**0.00 / 0.00 / 0.00**	1.00 / 1.00 / 0.97
D3 k-means	1.00 / 0.99 / 0.98	0.96 / 1.00 / 0.97	0.98 / 0.99 / 0.98	1.00 / 1.00 / 0.98
D4 スペクトル	1.00 / 0.97 / 0.85	**0.09 / 0.04 / 0.04**	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00

この表から読み取れる骨組みは，次の 4 つである。後の節で 1 つずつ根拠を確かめる。

- 符号反転は，どの検知器でもおおむね見つかる。Non-IID が強いと D1・D4 が少し下がる（§20，§19）。
- ALIE を，D2 は $z \leq 1.5$ でいつも**逆向き**に分ける（§15）。
- D1 は $z = 1.25$ だけで崩れ，その前後では見つかる（§17）。
- D4 は小さい z では逆向き，ある z を境に完全に見つかる。その境は Non-IID で動く（§18）。

14 主張：精度と収束

主張。 攻撃なしの精度は $\alpha = 1000/5/1$ で 98.0% / 97.4% / 94.3%。$\alpha = 1000, 5$ は 50 ラウンドでほぼ頭打ちだが，$\alpha = 1$ はまだ上がっている。

根拠。 `analysis/accuracy_table.csv`（`results.csv` から作る）と `accuracy_by_round.png`。収束の目安は，45 ラウンドから 50 ラウンドまでの精度の上がり幅（`gain_last_5`）である。$\alpha = 1000, 5$ はどの条件でも 0.3 ポイント未満，$\alpha = 1$ は攻撃なしで 0.53，ALIE で 0.63〜0.88 ポイント。【観測】

読むときの注意。

- $\alpha = 1$ の「最終 5 ラウンドの検知性能」は，学習が進みきる前の値である。ラウンド数を増やすと変わる可能性がある。
- 本番前の簡易な確認（Flower を通さない，1 試行）では上がり幅は 0.3 だった。本番の 10 試行の平均のほうが大きかった。
- $\alpha = 1$ の符号反転は約 76% で頭打ちになっている（上がり幅 0.05）。これは「収束した」のではなく，攻撃で学習が止まっている状態である。

15 主張：D2 は ALIE を逆向きに完全に分ける

主張。 D2（参照方向とのコサイン距離）の AUC は，$z \leq 1.5$ の ALIE で全 α とも 0.00。悪性が**いつも一番正常らしい**と判定されている。

根拠。

- AUC：`detection_final_auc_table.csv`。$z = 2$ で α により 0.83 / 0.12 / 0.00 と分かれ，$z = 3$ で 1.00 / 1.00 / 0.97。【観測】
- スコアそのもの：`detection_updates.csv` の `d2_cosine` 列を，`is_malicious` で分けて平均する。$z \leq 1.5$ の全ラウンドで，良性は 0.65〜0.89，悪性は 0.21〜0.23。悪性のスコアは z とともに上がる（$z = 0.385$ で 0.05，$z = 1.5$ で 0.44）。【観測】

なぜそうなるか。 【解釈】ALIE の悪性は「良性の平均 − 少し」なので，向きが良性の平均にとても近い。D2 の参照方向は全員の座標ごとの中央値で，これも平均に近い。だから悪性のほうが参照方向に近くなる。さらに，悪性 4

台は同じベクトルなので，中央値そのものが悪性の側に引き寄せられ，逆転が強まる。良性はそれぞれ自分のデータの方向に引っ張られるので，参照方向から離れる。

読むときの注意。 AUC 0 は「見分けられない」ではない。スコアの向きを逆にすれば完全に見分けられる。壊れているのは「スコアが大きいほど悪性」という片側だけの判定規則である。卒論では，この結果を Wang ら（2023）の指摘と結びつけている。Wang らは「パラメータの値が大きく違うのに，類似度では似ていると評価されてしまう」ことを示し，それを突く攻撃 Faker を作った。ALIE の悪性は，良性の平均に**実際に近い**ので似ていると評価される。仕組みは違うので，卒論では「同じ脆弱性が生じた」とは言わず，「類似度だけに頼る検知の限界という点で整合する」と書いている。【解釈】審査では「両側を見る検知器にすれば済むのでは」と聞かれうる。その答えは §17 の D1 が実例になる（D1 は「中央値からのずれの絶対値」なので両側を見ている）。

16　主張：D3 はいつも高いが，理由に注意

主張。 D3（k-means）は全条件で AUC 0.92 以上。z にも α にもほとんど依存しない。【観測】

なぜそうなるか。 【推測】ALIE の悪性 4 台は**まったく同じベクトル**を送る（原論文どおり）。特徴量の空間で 4 点が 1 点に重なるので，2 つに分けると，この固まりが切り出されやすい。D3 が ALIE の「巧妙さ」を見破っているのか，「4 台が同じ」という性質を拾っているだけなのかは，この実験では区別できない。

読むときの注意。 これは卒論で一番突かれやすい弱点の 1 つである（§25）。「同じ更新が複数届いたら怪しい」と調べるだけで ALIE は見つかってしまう。この攻撃の設定の限界として書く。確かめるには，悪性 4 台の更新に小さなばらつきを足した ALIE で D3 がどうなるかを見る必要がある（未実施）。

17　主張：重心に近すぎる悪性も目立つ（高次元の殻）

卒論の考察の中心である。直感に反するので，ていねいに説明する。

主張。 中心仮説は「悪性が良性の広がりの内側（push_ratio < 1）に入ると見つからなくなる」と予測した。しかし，push_ratio ≈ 0.39（$z = 0.385$）の ALIE は，D1・D3 で AUC 0.96 以上で見つかった。D1 が一番見つけにくかったのは $z = 1.25$ で，z をさらに上げると再び見つかった。

直感：低い次元と高い次元の違い。 2 次元で点をばらまくと，重心の近くにも点がある（§3.5 の図 2）。このとき，重心の近くに置いた悪性は，良性の中に紛れる。中心仮説は，この絵を前提にしていた。

ところが，次元がとても高いと事情が変わる。重心からの距離は，たくさんの座標のずれを 2 乗して足したものの平方根である。足し合わせる項が多いと，その合計はどの点でもだいたい同じ値になる（多数の値を足すと，ばらつきが平均に比べて小さくなるのと同じ理屈）。その結果，良性の点は「重心からほぼ同じ距離」に集まり，重心のまわりには**点がほとんどない**。良性は，中身の詰まった玉ではなく，薄い**殻**（球の表面の近く）に乗っている（図 7）。本実験の更新ベクトルは約 26 万次元である。

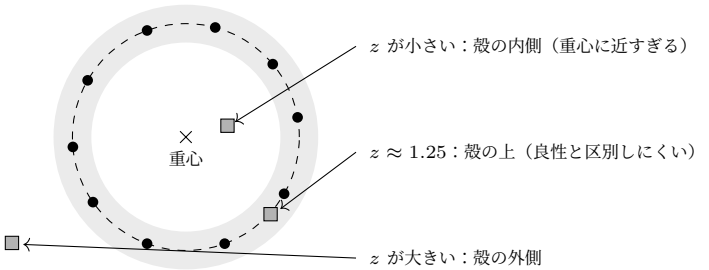


図 7　高い次元での良性の配置のイメージ（2 次元で描いた模式図）。黒い点（良性）は重心からほぼ同じ距離の殻（灰色の帯）に集まり，重心のまわりは空いている。灰色の四角が ALIE の悪性。

殻の内側にいる悪性は，「重心に近すぎる」という点で目立つ。殻の上に来たときが一番紛れ，殻の外に出るとまた目立つ。

根拠 1：良性は本当に殻に乗っているか。【観測】良性の重心からの距離を d_i とする。ALIE では，式から

$$\frac{\text{push_ratio}}{z} = \frac{d_i \text{ の 2 乗平均の平方根}}{d_i \text{ の平均}} = \sqrt{1 + \mathrm{CV}^2}$$

が成り立つ（CV は d_i の変動係数 = 標準偏差 ÷ 平均）。よって，ログの push_ratio と z から，CV = $\sqrt{(\text{push_ratio}/z)^2 - 1}$ と求められる。最終 5 ラウンドの ALIE の行で求めると，push_ratio/z は $\alpha = 1000/5/1$ で 1.003 / 1.004 / 1.016，CV は約 0.08 / 0.09 / 0.18 になる。つまり，良性の重心からの距離は，平均の ± 1〜2 割の範囲に集まっている。殻の厚さは，Non-IID が強いほど厚い。

根拠 2：ノルムの比。【観測】`detection_updates.csv` の `delta_norm` 列から，ラウンドごとに「悪性のノルムの平均 ÷ 良性のノルムの平均」を求め，最終 5 ラウンドで平均する（表 13）。

表 13　ALIE の悪性のノルム ÷ 良性のノルム（最終 5 ラウンド。3 つの α でほぼ同じ）と，D1 の AUC。

z	0.385	0.75	1	1.25	1.5	3
ノルムの比	0.37〜0.40	0.65〜0.66	0.84〜0.85	**1.03〜1.04**	1.21〜1.23	2.23〜2.31
D1 の AUC（$\alpha = 1000$）	1.00	1.00	0.88	**0.16**	0.99	1.00

D1 が一番崩れる $z = 1.25$ は，悪性と良性のノルムがほぼ揃う z と一致する。z が小さいときは「ノルムが小さすぎる」ことで，z が大きいときは「大きすぎる」ことで見つかっている。

根拠 3：予測が当たった。【予測で確認】以前の 54 回の実行は $z \leq 1$ しかなく，「D1 は z を上げると下がる」ところまでしか見えていなかった。殻の解釈が正しければ，「z が 1 をやや超えたところで最小になり，その先で回復する」はずである。これを再スイープの**前に**計画書に書き，結果は全 α で $z = 1.25$ が最小，$z \geq 1.5$ で回復となった。この節の主張が，卒論の中で一番強い根拠を持つ。

読むときの注意。

- 最小が $z = 1$ でなく 1.25 なのは，D1 が「重心からの距離」ではなく「原点からのノルム」を見ているためと考えられる。ノルムが揃う z が 1.25 だった（根拠 2）。【解釈】
- 「殻」は，距離やノルムだけで見たときの話である。向きを見る D2 は，殻の上（$z \approx 1$）を過ぎても $z = 2$〜3 まで逆向きのままだった。
- ALIE が見つかったのは，原論文の失敗ではない。原論文が避けようとしたのは Trimmed Mean や Krum であり，「重心に近すぎる更新を怪しむ」検知は想定していない（§3.4）。

18　主張：D4 の境界は Non-IID で動く（分布の形）

主張。D4（スペクトル）は，z が小さいと逆向き（AUC 0.04〜0.16），ある z を境に 1.0 近くへ切り替わる。その境は Non-IID が強いほど大きい z にずれる。これは，Non-IID が良性の広がりの**大きさ**ではなく**形**を変えるためと考えられる。

根拠 1：同じ push_ratio でも AUC が違う。【観測】$z = 0.75$ では push_ratio はどの α でも約 0.75 なのに，D4 の AUC は 0.998 / 0.385 / 0.161（$\alpha = 1000/5/1$）。push_ratio では説明できない違いである。

根拠 2：良性の広がりの形。【観測】`detection_structure_table.csv` の `pc1_ratio`（攻撃なし，最終 5 ラウンド）は 0.086 / 0.122 / 0.193。第 1 主成分の寄与率は，「良性のばらつきのうち，一番大きい 1 方向が何割を占めるか」である。値が大きいほど，良性は特定の方向に細長く広がっている（丸くない）。一方，広がりの大きさ（spread）は 0.143 / 0.143 / 0.155 で，ほとんど変わらない（§19）。

根拠 3：D4 の AUC と連動する量。【観測】`mal_cos_pc1_all`（悪性のずれの方向と，全員の更新の第 1 主成分の重なり，0〜1）は，$z = 0.75$ で 0.920 / 0.305 / 0.138 であり，D4 の AUC と並ぶ。ALIE の 24 条件で順位相関 0.92，ラウンド単位で 0.85。

なぜそうなるか。【解釈】D4 は「全員の更新が一番ばらついている方向」に射影して，そこで外れているものを怪しむ。悪性のずれがその方向になれば見つかり，ならなければ悪性の射影はほぼ 0 になって逆転する。Non-IID が強いと，良性自身が特定の方向に大きくばらつくので，悪性はその方向を「奪えず」，より大きな z が必要になる。

読むときの注意。

- 根拠 3 の `mal_cos_pc1_all` は，D4 の判定のしかたそのものに近い量である。「D4 が失敗する仕組み」を言い換えている面があり，独立な証拠としては弱い。
- α は 3 水準しかない。「形が原因」というのは対応関係であって，因果を示したわけではない。
- この解釈は，以前の 54 回の実行の結果を見たあとで立てたものである（予測で確認したものではない）。

19 主張：中心仮説は支持されなかった（符号反転と push_ratio）

主張。中心仮説は「Non-IID → 良性の広がりが大きくなる → push_ratio が下がる → 見つけにくくなる」という連鎖だった。符号反転で確かめたが，この連鎖は学習が進んだ段階では成り立たなかった。

根拠。【観測】

表 14　spread（攻撃なし）と符号反転の push_ratio。$\alpha = 1000/5/1$ の順。

	第 1〜5 ラウンド	最終 5 ラウンド	全ラウンド
spread	0.149 / 0.353 / 0.434	0.143 / 0.143 / 0.155	0.151 / 0.182 / 0.211
符号反転の push_ratio	8.62 / 2.85 / 1.96	1.58 / 2.03 / 2.55	—

- 学習の序盤は，仮説どおり Non-IID で spread が約 2.4〜2.9 倍になり，push_ratio も下がる。
- 学習が進むと，spread の差は 1.08 倍まで縮み，push_ratio の大小関係は**逆転**する。最終 5 ラウンドで 1 を下回った試行はない。
- push_ratio と AUC の順位相関（`detection_pushratio_corr_table.csv`）は，全体で -0.32〜-0.02，α ごとでも弱いか負。例外は $\alpha = 1$ の D1（0.69）だけ。

それでも Non-IID は検知を難しくしている。【観測】符号反転の D4 は 1.00 / 0.97 / 0.85，D1 は 0.97 / 0.94 / 0.88 と，Non-IID で下がる。ただし，その原因は push_ratio ではない。$\alpha = 1$ では，符号反転のずれの方向と良性の第 1 主成分の重なり（`mal_cos_pc1`）が 0.403 と大きく（他は 0.051 / 0.307），悪性が良性の主なばらつきの方向に重なって目立ちにくくなった可能性がある。【推測】

読むときの注意。

- 以前の 54 回の実行（データ数も偏る分け方）では，符号反転が最も Non-IID な条件で AUC 0.55〜0.65 まで落ちた。データ数を揃えた本実験では，そこまで落ちない。以前の低下にはデータ数の偏りが関わっていた可能性がある。ただし 2 つの分け方はラベルの偏り方も違うので，断定はできない。【推測】
- 仮説が外れたこと自体は，隠さずに報告するのが正しい。卒論では「なぜ外れたか」（§17）と「代わりに何が効いていたか」（§18）まで書けている。

20 主張：符号反転を D1 が見つけられる理由

主張。符号反転は向きを逆にするだけでノルムは変えないのに，D1（ノルム）で見つかる（0.97 / 0.94 / 0.88）。これは，学習の途中で悪性のノルムが大きくなっていくためである。

根拠。【観測】`detection_updates.csv` から求めた「悪性のノルム ÷ 良性のノルム」は，第 1〜5 ラウンドで 1.00 / 1.07 / 1.15，最終 5 ラウンドで 1.30 / 1.65 / 2.18。D1 の AUC も，第 1〜5 ラウンドの 0.50〜0.57 から，第 31〜50 ラウンドの 0.87〜0.93 へ上がる（`detection_auc_by_round.png` の右端の列）。

なぜそうなるか。【推測】符号反転を受け続けたグローバルモデルは，悪性クライアントのデータにうまく合わなくなる。すると悪性クライアントは，自分のデータで学習するたびに大きく重みを動かすので，更新（を反転したもの）

のノルムが大きくなる。つまり D1 は，攻撃そのものではなく，攻撃が学習を通じて残した「副作用」を見ている。

読むときの注意。 ノルムの比は $\alpha = 1$ で一番大きいのに，D1 の AUC は $\alpha = 1$ で一番低い。良性どうしのノルムのばらつきも，Non-IID が強いほど大きいためと考えられる。【解釈】

21 主張：良性の各座標は，裾が正規分布より重い

主張。 ALIE の原論文は，良性の各座標が正規分布に従うと仮定している。本実験では，中心付近ではその仮定がよく当てはまるが，裾（極端な値）は正規分布より重く，Non-IID が強いほど重い。

根拠。 【観測】`mal_rank_mean` は，「悪性の値以下の良性が何割いるか」を座標ごとに求めて平均したもの。正規分布なら $\Phi(-z)$ になる（Φ は標準正規分布の累積分布関数）。

- $z \leq 1$：実測 0.34 / 0.30 / 0.22 / 0.17（$z = 0.385/0.5/0.75/1$）は，$\Phi(-z) = 0.350/0.309/0.227/0.159$ とよく合う。
- $z = 3$：$\Phi(-3) \approx 0.001$ なのに，実測は 0.073 / 0.110 / 0.180（$\alpha = 1000/5/1$）。裾の外まで押したつもりでも，良性の 1 割前後が悪性より外にいる。

読むときの注意。 この主張は卒論の結論には直接使っていないが，「原論文の仮定は Non-IID ではそのまま成り立たない」ことの具体例として，妥当性の議論で使っている。

22 主張：ALIE は FedAvg ではほぼ無害

主張。 ALIE は $\alpha \geq 5$ ではどの z でも精度をほとんど下げない（攻撃なしとの差は最大 0.5 ポイント）。$\alpha = 1$ で $z \geq 2$ のときだけ，はっきり下がる（$z = 2$ で約 2.6，$z = 3$ で約 5.2 ポイント）。【観測】

なぜそうなるか。 【解釈】20 台を単純に平均すると，悪性 4 台が $\mu - z\sigma$ を送ったときの平均のずれは，座標ごとに $0.2\,z\,\sigma$ になる。一方，良性 16 台の平均そのものも，偶然 $\sigma/\sqrt{16} = 0.25\,\sigma$ くらいは揺れる。$z \leq 1$ のずれは，この自然な揺れ以下なので，学習にはほとんど効かない。原論文も，単純な平均ではこの攻撃が薄まると報告している。

読むときの注意。 これは卒論の一番大きな弱点につながる。「害がほとんどない攻撃を見つけて何の意味があるのか」と聞かれうる（§25）。

23 事前の予測と結果

再スイープの前に計画書に書いた 3 つの予測と，その結果。

予測	結果	根拠
D1 は z が 1 をやや超える付近で最小になり，それより大きい z で再び上がる	当たった	全 α で $z = 1.25$ が最小，$z \geq 1.5$ で回復（§17）
D4 は $\alpha = 1$ でも z を大きくすれば見つけるようになり，その z は Non-IID が強いほど大きい	当たった	$\alpha = 1$ でも $z \geq 1.25$ で 0.99 以上。境は $\alpha = 1000$ で 0.5〜0.75，$\alpha = 5, 1$ で 0.75〜1（§18）
$z \geq 2$ では FedAvg でも ALIE が精度を下げうる	一部	$\alpha = 1$ だけ（§22）

なお，$z = 3$ の 10 ラウンドだけの動作確認（全検知器 AUC 1.0）は予測を書く前に見ている。計画書にもそう記録してある。

24 数値を自分で計算し直す

卒論の数値は，次のどれかで再現できる。

1. **分析スクリプトの出力**：`python analyze_detection.py experiments_detection_v3/_logs` を実行すると，`analysis/` に図と表ができる。最終 AUC（`detection_final_auc_table.csv`），精度（`accuracy_table.csv`），

構造指標（detection_structure_table.csv），符号反転の相関（detection_pushratio_corr_table.csv）はここから取る。

2. **ノルムの比と D2 のスコア**：detection_updates.csv から，次のように計算する（プロジェクト直下で実行）。

```
import pandas as pd
from analyze_detection import keep_last_runs
u = pd.read_csv("experiments_detection_v3/_logs/detection_updates.csv")
u = keep_last_runs(u, "updates")            # 回し直した実行の古い行を捨てる
a = u[u["attack_type"] != "none"]
# ラウンドごとに，悪性と良性のノルムの平均を出し，その比をとる
g = a.groupby(["alpha", "attack_type", "alie_z", "trial", "round", "is_malicious"],
              dropna=False)["delta_norm"].mean().unstack("is_malicious")
g["ratio"] = g[1] / g[0]
g = g.reset_index()
last = g[g["round"] > 45]                    # 最終 5 ラウンド
print(last.groupby(["alpha", "attack_type", "alie_z"], dropna=False)["ratio"].mean())
# D2 のスコア：良性と悪性で平均を比べる
al = a[(a["attack_type"] == "alie") & (a["alie_z"] <= 1.5)]
print(al.groupby(["alpha", "is_malicious"])["d2_cosine"].mean())
```

3. **殻の厚さ（CV）**：detection_metrics.csv の ALIE の行で，最終 5 ラウンドの push_ratio/z を α ごとに平均し，$\sqrt{(\cdot)^2-1}$ を計算する（§17 の根拠 1）。
4. **ラウンドの区間ごとの AUC**：detection_metrics.csv の round を 1〜5，6〜15，16〜30，31〜50 に分け，（攻撃，α，検知器）ごとに auc を平均する。卒論の「第 1〜5 ラウンドの平均」はこれである。

25 審査で聞かれそうなことと答え方

Q1. ALIE は FedAvg ではほぼ無害なのに，検知を調べる意味はあるのか。答えの方向：本研究は検知を，集約手法とは独立した「前段」として評価している。検知できれば，除外や重み下げに使える。ALIE は Trimmed Mean や Krum を破るための攻撃なので，そうした集約手法と組み合わせる場面では検知の価値が大きい。ただし，その組み合わせでの効果は確かめていない（今後の課題）。

Q2. D3 が ALIE を見つけるのは，悪性 4 台が同じベクトルだからではないか。答えの方向：その可能性はある，と認める（§16）。同じベクトルを送るのは原論文どおりだが，現実の攻撃者は小さなばらつきを足して避けられる。確かめるには，ばらつきを足した ALIE の追加実験が必要。

Q3.「Non-IID は形を変える」は後付けではないか。答えの方向：そのとおり，後付けの解釈である（§18）。α も 3 水準しかない。一方，「重心に近すぎても目立つ」（殻）の解釈は，実験の前に予測を書いて当たっている（§17）。2 つの主張の確かさは違う。

Q4. $\alpha = 1$ は収束していないのに，その検知性能を議論してよいのか。答えの方向：収束していないことは明記している（§14）。ラウンド推移の図で，学習の段階による変化も示している。ラウンドを増やした確認は今後の課題。

Q5. 検定をしていないのはなぜか。答えの方向：同じ実行の中のラウンドどうしは独立でないので，ラウンドを点として検定すると有意に出すぎる。そのため，試行（10 回）を単位にした平均と標準偏差で示し，傾向の記述にとどめた。必要なら試行単位のブートストラップで信頼区間を出せる。

Q6. 中心仮説が外れたのは研究の失敗ではないか。答えの方向：外れた理由（高次元では良性が殻に乗る）と，代わりに効いていた要因（良性の広がりの形，検知器の判定の向き）を示せている。仮説を後から書き換えず，外れたこととして報告している点も，研究の進め方として正しい。

第 V 部
リファレンス

26　攻撃と検知器の早見表

考え方と式の説明は，§3 にある。ここは，実行時に見返すための一覧である。

26.1　攻撃

attack-type	動く場所	内容
none	（なし）	攻撃なし。悪性 0 台として扱う。良性の広がりの基準値を得るための条件。
sign_flip（A1）	クライアント側（client_app.py）	更新量 Δ = 学習後 − 学習前 の符号を反転して送る。向きが正反対になり，大きさは変わらない。
alie（A2）	サーバ側（robust_strategy.py）	良性の更新の，重みごとの平均 μ と標準偏差 σ から $g^{\mathrm{mal}} = \mu - z\sigma$ を作り，悪性クライアント全員に同じ値を割り当てる。良性のばらつきの範囲内に収まるので，見つかりにくい。

ALIE は Baruch ら（“A Little Is Enough”, 2019）の生成則に基づく。z は実験条件として与え，原論文 Algorithm 3 の $z_{\max}$（20 台・悪性 4 台で約 0.385）は参照値として使うにとどめる。悪性クライアントは，データ数を偽らずに申告するものとする（FedAvg の重みは，申告されたデータ数である）。

26.2　検知器

記号	スコア	得手・不得手
D1 d1_norm	$\lvert \lVert g_i \rVert - \mathrm{median}_j \lVert g_j \rVert \rvert$	更新の大きさを見る。極端に大きい・小さい更新に強く，大きさが良性と変わらない攻撃に弱い。
D2 d2_cosine	$1 - \cos(g_i, r)$，r は重みごとの中央値	更新の向きを見る。向きを逆にする符号反転に強い。
D3 d3_kmeans	標準化した 4 つの特徴（大きさ，D2，第 1・第 2 主成分）で 2 グループに分け，多数派の中心までの距離	大きさと向きをまとめて見る。悪性が多数を占めると成り立たない。
D4 d4_spectral	$(\tilde{g}_i \cdot v)^2$，v は第 1 主成分の方向	一番ばらつきの大きい方向への突き出しを見る。良性どうしのばらつきのほうが大きいと，悪性を見つけられない。

D1 の同点について。クライアントの台数が偶数のとき，中央値は真ん中の 2 台の平均になるので，その 2 台のスコアは数式の上で必ず等しくなる。計算の丸め誤差でこれがずれないよう，ごく近い値は同じ値にそろえている。ROC-AUC では，同点を「半分正解」と数える。

27　エラーで止まる条件

設定の誤りがあるとき，黙って想定と違う実験が走るのを防ぐため，次の場合は学習の途中でもエラーで止まるようにしてある。止まった条件は final_model.pt が作られないので，スイープでは「失敗」として記録される。

止まる条件	理由
`pyproject.toml` に，コードが読む項目が無い	設定の書き忘れ。別の値で黙って動くことはない。
`attack-type`，`aggregation-method`，`data-split` に，使えない値を指定した	現在使えるのは，それぞれ `none` / `sign_flip` / `alie`，`fedavg`，`dirichlet_equal` / `dirichlet` だけ。
`num-clients` × `malicious-ratio` が整数にならない	丸めると，記録される割合と実際の台数がずれるため（例：20 台 × 0.125 = 2.5 台）。
学習結果を返したクライアントの数が `num-clients` と違う	起動している台数と設定が食い違うと，悪性に選ばれた番号の一部が実在せず，悪性の台数が設定より少なくなるため。
検知のログに書く行に，項目が欠けている	空欄のまま記録されるのを防ぐため。
Flower のコマンドが見つからない（スイープ開始時）	仮想環境の Python で実行していない。1 条件も回さずに止まる。
GPU の設定がマシンと食い違っている（スイープ開始時）	クライアントが起動できない，または，GPU で回しているつもりでクライアントが CPU で学習する，という状態を防ぐため（§7.3）。
`device` に，そのマシンで使えない装置を指定した	指定した装置で回っていない結果が，黙って記録されるのを防ぐため。

28 再現性

同じ装置で同じ条件を回し直すと，ログの数値は最後の桁まで一致する（CPU と，Apple Silicon の Mac の GPU で確認）。そのために，次のことをしている。

- `trial` 番号を乱数の種として，データ分割，モデルの初期値，学習時のデータの並べ替えを固定する（初期値は `server_app.py`，並べ替えは `client_app.py` で `torch.manual_seed` により固定）。
- 悪性クライアントの選出は `malicious-seed` で固定する（スイープでは `trial` と同じ値を渡す）。
- サーバは，クライアントからの返信を，届いた順ではなくクライアント番号の順に並べ直してから計算する。

`trial` を変えると，データの分け方，モデルの初期値，悪性になるクライアントのすべてが変わる。

装置による違い。CPU と GPU では計算の丸め方が違うので，同じ条件でも結果の数値は変わる（AUC が変わるラウンドもある）。1 つのスイープの中では，装置を変えないこと。どの装置で回したかは，`run_config.txt` の `device（effective）` に記録される。

NVIDIA の GPU での再現性は未確認。NVIDIA の GPU では，結果が変わりにくくなる設定（`cudnn.deterministic`）を入れてあるが，回し直したときに完全に一致するかは，まだ確かめていない。確かめるには，同じ条件を，出力先を変えて 2 回実行し，2 つの `detection_metrics.csv` を比べる（`run_id` の列は実行の日時を含むので，それ以外の列を比べる）。

29 トラブルシューティング

症状	対処
`flwr: command not found`，または「Flower のコマンドが見つかりません」	仮想環境が有効になっていない。`source .venv/bin/activate` を実行する。
「学習結果を返したクライアントは N 台ですが，設定の num-clients は M です」	起動している台数と設定が合っていない。手で `flwr run` を実行しているなら，`--run-config` の `num-clients` を，`~/.flwr/config.toml` の `num-supernodes` に合わせる。
「N 台 × 割合 r = x 台で，整数になりません」	`malicious-ratio` を，台数との積が整数になる値にする（20 台なら 0.1，0.2，0.3 など）。
スイープで条件が「FAILED」になる	画面に出力の末尾が表示され，`failures.csv` にも記録される。原因を直してからもう一度実行すれば，失敗した条件だけが回る。
`detection_metrics.csv` の `auc` が NaN	攻撃なしの条件（悪性 0 台）なら正常。
「client-num-gpus = … ですが，このマシンでは NVIDIA の GPU（cuda）が使えません」	GPU の無いマシンなら，`client-num-gpus` を 0.0 にする。GPU があるはずなら，PyTorch が GPU 対応の版かを確かめる（§7.3）。
「NVIDIA の GPU が使えるマシンですが，client-num-gpus = 0.0 のため…」	GPU で回すなら `client-num-gpus` を 0 より大きくする。CPU で回すなら `device` を `"cpu"` にする。
`CUDA out of memory`	同時に GPU を使うクライアントが多すぎる。`client-num-gpus` を大きくする（例：0.5）。
実行が始まったまま，いつまでも進まない	GPU の無いマシンで `client-num-gpus` が 0 より大きいと，クライアントが起動できずに待ち続ける。手で `flwr run` を実行した場合に起こりうる（スイープでは開始前に止まる）。
MNIST のダウンロードの警告	`HF_TOKEN` が未設定という警告は無視してよい。
メモリ不足	更新ベクトルは（クライアントの台数 × 266,060）の表としてサーバ上に載る。`num-clients` を下げる。

30 設定を決めた経緯と，結果を読むときの注意

2026 年 10 月 11 日に，実験の設定と分析の方法をいくつか変更した。あとから「なぜこの値なのか」「この結果はどう読めばよいのか」が分からなくならないように，決めたことと理由をここにまとめる。正式な記述は研究計画書（`sotsuron_plan2.tex`）の「データ分布」と「再スイープ」の節にある。

30.1 決めたことの一覧

項目	決めた内容	理由（ひとことで）
データの分け方	`dirichlet_equal`（全員 1,500 枚）	枚数の偏りを無くし，数字の偏りだけの影響を見るため。
Dirichlet の式	$\mathrm{Dir}(\alpha \cdot p)$	Hsu ら（2019）の式そのままにするため。独自の変更はしない。
α の値	1000 / 5 / 1	式を直す前の 100 / 0.5 / 0.1 と，まったく同じ分け方になる値。
一番大きい α	100 ではなく 1000	IID の比較対象として使うため。100 だと少し偏る。
ローカルエポック	1 から 2 に変更	1 台あたりの枚数が半分になり，学習が足りなくなったため。
ラウンド数	50 のまま	「最後の 5 ラウンド」で評価する決まりを変えないため。
push_ratio の分析	符号反転だけ	ALIE では push_ratio が z と同じ量になるため。
出力先	`experiments_detection_v3`	以前の 54 回の実行と混ぜないため。

30.2 α の定義を Hsu らの式に合わせた

枚数をそろえる分け方に切り替えたとき，最初は，以前の α の値（100 / 0.5 / 0.1）をそのまま使えるように，ディリクレ分布に渡す値を $\alpha \times 10 \times p$ としていた（p はデータ全体での各数字の割合）。しかし，Hsu らの式は $\alpha \times p$ であり，これでは α の意味が 10 倍ずれる。「原論文にない独自の変更は入れない」という方針で，式を $\alpha \times p$ に直した。

式を直すと，同じ偏り具合を表す α の値は 10 倍になる。そこで α を 1000 / 5 / 1 とした。この値で作られる分け方は，直す前の 100 / 0.5 / 0.1 のときと，`trial` 1〜10 のすべてで完全に同じであることを確かめてある。変わったのは，式と α の書き方だけである。

α の値をそのまま（100 / 0.5 / 0.1）にしなかった理由。 式だけ直して値を変えないと，偏りが今よりずっと強くなる。一番多い数字が占める割合は，100 / 0.5 / 0.1 の順に 16% / 75% / 91% になり，$\alpha = 0.1$ ではほぼ「1 台が 1 種類の数字だけを持つ」状態になる。これでは 50 ラウンドで学習が収束しにくい。

一番大きい α を 1000 にした理由。 この水準は，Non-IID と比べるための「IID」として使う。表 15 のとおり，$\alpha = 1000$ は，でたらめに 1,500 枚を配った場合とほとんど区別がつかない。$\alpha = 100$ はばらつきが約 4 倍あり，「弱い Non-IID」になってしまう。論文では，$\alpha = 1000$ を「≈IID」と書く。

表 15　α が大きいときの偏り具合と，完全な IID との比較。

	一番多い数字の割合	比率の標準偏差
でたらめに 1,500 枚を配った場合（完全な IID）	11.6%	約 0.008
$\alpha = 1000$	12%	約 0.009
$\alpha = 100$	16%	約 0.030

Hsu らと違うところ。 式は同じだが，次の 3 点は違う。論文には明記する。

- p は，Hsu らでは「どの数字も同じ割合」である。本実験では，MNIST の実際の割合（9.0%〜11.2%）を使っている。
- Hsu らは 100 台に 500 枚ずつ，全データを配る。本実験は 20 台に 1,500 枚ずつで，全 60,000 枚の半分だけを使う。全部を配ると在庫切れが増え，引いた比率どおりに配れなくなるためである（$\alpha = 1$ で，ずれが約 2% から約 23% に広がる）。

- ある数字の在庫が切れたときの配り方は，Hsu らには書かれていない。「切れた数字を除いて比率を作り直す」は，本実験で決めたものである。

30.3 ローカルエポックを 2 にした

枚数をそろえる分け方では，1 台が学習に使う枚数が，以前の平均約 2,400 枚から 1,200 枚に減る。1 ラウンドで重みを更新する回数が半分になるので，エポック 1 のままでは 50 ラウンドで学習が足りなかった。攻撃なし・`trial` 1 で 50 ラウンド回して確かめた結果が，表 16 である（Flower を通さない簡易な実行）。

表 16　ローカルエポックと，50 ラウンド時点のテスト精度。

	$\alpha = 1000$	$\alpha = 1$	300 回の実行時間
エポック 1	97.0%	92.2%	基準の約半分
エポック 2（採用）	98.0%	95.1%	基準
エポック 3	98.3%	96.1%	約 1.5 倍
以前の 54 回の実行（エポック 1）	98.1%	95.9%	—

- エポック 2 にすると，1 ラウンドに見る枚数が 2,400 枚に戻り，以前の実行と学習の量がそろう。精度も以前に近い。
- エポック 3 は，$\alpha = 1$ の精度が約 1 ポイント上がるだけで，実行時間が 1.5 倍になり，以前の実行と学習の量がそろわなくなる。
- ラウンド数を 100 にする案は，「最後の 5 ラウンド」で評価する決まりや，以前の実行との対応が変わるので，採らなかった。

注意。 この簡易な確認でも，エポック 2 の $\alpha = 1$ は 45 ラウンドから 50 ラウンドのあいだに 0.3 ポイント上がっていた。本番の 300 回の実行で確かめると，$\alpha = 1$ の上がり幅は，攻撃なしで 0.53 ポイント，ALIE で 0.63〜0.88 ポイントであり，50 ラウンドでは**収束していなかった**。$\alpha = 1000$ と $\alpha = 5$ は 0.3 ポイント未満で，ほぼ頭打ちだった。論文では，$\alpha = 1$ の最終 5 ラウンドの検知性能を「学習が進みきる前の段階のもの」と書き，精度と検知性能のラウンド推移の図を添える（`accuracy_by_round.png`，`accuracy_table.csv`）。

30.4 結果を読むときの注意

AUC が 0 に近いのは，「見分けられない」ではない。 AUC が 0.5 なら見分けられていないが，0 に近いなら，「悪性のほうが，いつも良性らしく見えている」ということである（§4.1）。以前の 54 回の実行では，ALIE に対する D2 の AUC が全部の条件で 0.00 だった。D4 も，z が小さいと 0.01〜0.08 だった。ALIE の悪性の更新は良性の平均のすぐ近くに作られるので，「外れているものを探す」検知器には，だれよりも普通に見える。したがって，「AUC が低い＝検知が難しい」とまとめてはいけない。0.5 からどれだけ離れているかと，どちら向きかを分けて書く。

ALIE の悪性 4 台は，まったく同じ更新を送っている。 これは原論文（Baruch らの Algorithm 3）のとおりであり，実装の誤りではない。ただし，次の 2 点に注意する。

- D3（k-means）が ALIE をよく見つけるのは，同じ 4 点が固まって見えるためかもしれない。ALIE の巧妙さを見破っているとは限らないので，考察で確かめる。
- 「同じ更新が複数届いたら悪性」と調べるだけで見つかってしまう。これは，この攻撃の設定の限界として書く。

ALIE では，push_ratio は z と同じ量である。 §3.5 に書いたとおり，ALIE では push_ratio がほぼ z になり，α を変えても変わらない（以前の実行での push_ratio $\div z$ は 1.00〜1.14）。そこで，push_ratio と AUC の関係は符号反転だけで調べ，ALIE は z を横軸にした図（`detection_auc_by_z.png`）で見る。同じ z なのに α によって AUC が違えば，それは push_ratio では説明できない違いである。

符号反転の push_ratio は，ほとんど α で決まる。 符号反転には強さを決める数値がないので，push_ratio を動かしているのは主に α である。そのため，全部の点で push_ratio と AUC に相関があっても，「α が違うと AUC が違う」ことを言い換えただけかもしれない。以前の実行で試すと，順位相関係数は全体では 0.68〜0.74（D2〜D4）

だったが，同じ α の中だけで見ると -0.04〜0.48 だった。再スイープでは，同じ α の中でも連動しているかを，`detection_pushratio_corr_table.csv` で確かめる。

以前の 54 回の実行とは，数値を直接比べない。 データの分け方，α の定義，ローカルエポックの 3 つが違う。同じ図や表には並べず，「変更の根拠になった予備的な実行」として扱う。

30.5 原論文との違いのまとめ

表 17　ALIE について，原論文（Baruch ら，2019）と本実験の違い。

項目	原論文	本実験
式	$\mu + z\sigma$	$\mu - z\sigma$（左右対称なので同じ考え方）
μ，σ の求め方	悪性ワーカー自身のデータから推定	良性 16 台の更新から計算（より強い攻撃者）
z	$z_{\max}$ を計算して使う	8 通りの値を与える（$z_{\max} \approx 0.385$ を含む）
悪性の更新	全員同じ	全員同じ（原論文どおり）
規模	51 台中 12 台	20 台中 4 台
何に適用するか	1 ステップの勾配	複数エポック学習したあとの更新

原論文は，防御のない単純な平均では攻撃の効果が薄まる，と報告している。本実験でも，FedAvg のもとでは，z の小さい ALIE がモデルに与える害は小さいはずである。本実験は，ALIE を「モデルをどれだけ壊すか」ではなく，「見つけられるか」を調べる対象として使っている。

30.6 本番の前に確かめたこと

- テスト 5 本（`test_*.py`）は，すべて合格した。
- Flower を通して 1 条件（$\alpha = 1$，ALIE，$z = 1$）を 2 ラウンド回し，5 つのログと `final_model.pt` が出力されることを確かめた。
- 一番強い攻撃（$\alpha = 1$，ALIE，$z = 3$）を 10 ラウンド回した。学習は壊れず（10 ラウンドで精度 81.6%），4 つの検知器の AUC はすべて 1.0 だった。検知できるかどうかが分かれるのは，z が 0.75〜1.5 のあたりになりそうである。
- 上の 10 ラウンドは 107 秒かかった（Mac，1 回だけの計測）。ここから見積もると，1 回の実行（50 ラウンド）が約 8 分，300 回で約 40 時間である。実際の本番は，別の計算機の CPU で約 62 時間（1 回あたり約 11 分）かかった。実際に回す計算機で数条件を回し，時間を測ってから全体を流すこと。

30.7 あとで直すことにしたもの

どちらも，ログから図や表を作り直すだけなので，実験のあとで対応できる。

- `detection_auc_by_round.png` は，攻撃ごとに列を作る。z が 8 通りあると 9 列になり，論文には横に長すぎる。載せる z を 3〜4 個に絞る。
- `detection_final_auc_table.csv` は，以前はラウンドと `trial` をまとめて平均するだけで，ばらつきが分からなかった。2026 年 10 月 15 日に，`trial` ごとに最後の 5 ラウンドを平均してから，`trial` のあいだの平均と標準偏差を出す形に直した（同じ実行の中のラウンドどうしは似た値になるので，まとめて標準偏差を取ると，ばらつきを小さく見積もってしまう）。列は検知器ごとに `_mean` と `_std` の 2 つずつである。

31 本書が扱わないもの

次のものは，実験の実行には使うが，本書では詳しく説明していない。

- `test_*.py`：テスト。§9 の手順 1 で使う。

- `tools/verify_dirichlet_partition.py`：Dirichlet 分割の様子を図にして確かめるスクリプト。`python tools/verify_dirichlet_partition.py --alpha 1000 5 1 --clients 20 --trial 1` と実行すると，クライアントごとの枚数と保有クラスの要約が表示され，ヒートマップが `results_detection/` に保存される。以前の方式を確かめるときは `--split dirichlet` を付ける。
- `unused_features/`：現在の実験で使っていない機能の退避先（Adaptive / Scale 攻撃，Median / Trimmed Mean / Krum / Bulyan，`iid` / `non_iid` 分割，旧モデル，固定画像の保存，悪性クライアントの手動指定）。内容と戻し方は，同フォルダの `README.md` にある。
- `legacy_fang_reproduction/`：以前の Fang 攻撃再現実験の記録。本実験のコードからは参照していない。

32 用語集

ALIE
: 良性のばらつきの範囲内に隠れる攻撃。良性の平均から，標準偏差の z 倍だけずらした値を送る（§3.4）。

AUC（ROC-AUC）
: 検知器の成績。悪性 1 台と良性 1 台を選んだとき，悪性のスコアのほうが高い確率（§4.1）。

CSV
: 値をカンマで区切って並べたテキストファイル。1 行目が列の名前。

Dirichlet 分割
: ディリクレ分布を使って，データをクライアントに偏らせて配る方法。偏りの強さを α で調整する（§2.4）。

FedAvg
: 連合学習の基本的な集約の方法。データ数で重み付けした平均（§2.3）。

Flower（`flwr`）
: 連合学習を実行するためのライブラリ。サーバとクライアントのやり取りを受け持つ。

GPU
: 画像処理用の計算装置。ニューラルネットワークの計算を速く行える。NVIDIA のものは CUDA という仕組みで使う。

IID / Non-IID
: クライアントごとのデータの構成が，似ている（IID）／偏っている（Non-IID）。

MNIST
: 手書き数字（0〜9）の画像のデータセット。学習用 60,000 枚，テスト用 10,000 枚。

push_ratio
: 悪性が良性の重心から離れている距離を，良性の広がりで割ったもの。1 より小さいと，悪性は良性の集団の内側にいる（§3.5）。

SGD
: 確率的勾配降下法。データの一部（バッチ）で勾配を計算し，重みを少しずつ更新する方法。

trial
: 同じ条件を，乱数だけを変えて繰り返すときの番号。乱数の種として使う。

重み（パラメータ）
: モデルの中の，学習で調整される数値。本実験では 266,060 個。FedAvg の「重み付き平均」の重みとは別のもの。

エポック
: 手元のデータを 1 周すること。

学習率
: 1 回の更新で，重みをどれだけ動かすかの倍率。

仮想環境
: プロジェクトごとに分けた Python の環境。パッケージの版が，ほかのプロジェクトとぶつからないようにする。

クライアント

連合学習で，データを持ち，学習を行う側。

グローバルモデル

サーバが持つ，全体で共有するモデル。

勾配

各重みを動かしたときに，損失がどちらにどれだけ変わるかを並べたもの。

検知器

各クライアントの更新に，悪性らしさのスコアを付けるもの。本実験では D1〜D4 の 4 つ。

更新ベクトル

学習でモデルの重みがどれだけ動いたか。届いた重みから，配った重みを引いたもの（§3.2）。

コサイン

2 本のベクトルの向きの近さ。同じ向きで 1，直角で 0，正反対で -1。

サーバ

連合学習で，モデルを配り，集約する側。

集約

クライアントから返ってきた重みを，1 つにまとめること。

主成分（第 1 主成分）

データが一番大きくばらついている方向（§3.10）。

条件

1 回の実行の設定の組み合わせ（α，攻撃，z，trial など）。

スイープ

設定をいくつかの値に振って，その組み合わせをすべて実行すること（§4.3）。

spread（広がり）

良性クライアントの更新が，その重心からどれだけ散らばっているか。

損失

モデルの答えが，正解からどれだけ外れているかを表す数値。小さいほどよい。

種（seed）

乱数の出方を決める数値。同じ種なら，毎回同じ乱数が出る。

中央値

値を大きさの順に並べたときの，真ん中の値。極端な値に引っ張られにくい。

ノルム

ベクトルの長さ（大きさ）。

バッチ

学習で，一度にまとめて使うデータ（本実験では 32 枚）。

標準偏差

値のばらつきの大きさを表す数値。

符号反転

更新ベクトルの向きを正反対にする攻撃。

モデルポイズニング

悪性クライアントが，返す重みを書き換えて，学習を妨げる攻撃。

ラウンド

「配る → 学習する → 返す → 集約する」の一巡。

悪性クライアント / 良性クライアント

攻撃を行うクライアント / 正直に学習するクライアント。